CHAPTER 1

INTRODUCTION

1.1 Background and Context

Educational institutions such as universities, colleges, and schools operate on a daily basis with a wide variety of infrastructure facilities classrooms, laboratories, libraries, hostels, internet connectivity, furniture, and essential utilities like water and electricity. Maintaining these facilities in good condition is crucial for ensuring a productive learning and working environment for students, faculty, and administrative staff. However, managing infrastructure across a large campus presents significant challenges.

Traditional systems for reporting issues typically involve verbal complaints, paper forms, or emails sent to maintenance staff. These methods are fragmented and lack real-time tracking or accountability. Complaints can get lost, duplicated, or ignored, leading to frustration among users and delays in resolving problems. Without a centralized system, administrators find it difficult to monitor the status of issues, allocate resources efficiently, and analyse recurring problems.

As digital transformation spreads across all sectors, educational institutions are increasingly adopting technology to improve operational efficiency. From learning management systems to online admission portals, campuses are moving toward paperless, transparent, and automated workflows. Yet, one crucial area — infrastructure and facility issue management — often remains underdeveloped. There is a strong need for a modern, web-based platform that bridges the gap between problem reporting and resolution.

1.2 Purpose of the Project

The **Fix My Campus** project has been conceived to address this gap. It is a web application designed to provide a centralized platform where students and staff can easily report problems, track their status, and receive timely updates, while administrators can efficiently manage and resolve these issues. By digitizing the reporting and resolution process, the system promotes transparency, reduces delays, and enhances accountability.

The application enables:

- **User Authentication** for secure login, signup, and password recovery.
- **Issue Reporting** with details such as category, description, and location.
- **Issue Tracking** so users can see whether their complaint is pending, in progress, or resolved.

- **Admin Dashboard** for administrators to view all reported issues, filter them by category or status, update their progress, and manage users.
- **Visual Analytics** using Chart.js to show trends, categories, and counts of issues over time.
- **Audit Logs** to maintain a digital record of actions taken.

1.3 Problem Statement

Institutions without a centralized complaint management system face:

- Delays in identifying and addressing issues.
- Difficulty in prioritizing maintenance requests.
- Lack of transparency about which issues are resolved.
- Duplication or loss of complaints.
- No structured data for decision-making or resource planning.

Fix My Campus directly tackles these problems by offering an integrated platform where all stakeholders interact in a streamlined manner.

1.4 Objectives of the Project

The key objectives of the **Fix My Campus** project is:

1. To design and develop a web-based system for reporting, tracking, and resolving campus infrastructure and facility issues.
2. To provide a user-friendly interface for students and staff to submit and monitor complaints.
3. To empower administrators with dashboards, filtering options, and data analytics for informed decision-making.
4. To ensure transparency and accountability through audit logs and status tracking.
5. To create a scalable, secure, and responsive platform that works seamlessly on desktops, tablets, and smartphones.

1.5 Scope of the Project

The project covers:

- **User Module**: Registration, login, profile management, reporting issues, viewing status of submitted issues.

- **Admin Module**: Authentication, issue management (view, filter, update status, delete), user management (ban/unban users), and viewing audit logs.
- **Analytics Module**: Displaying charts for issue trends by category and status.
- **Database Integration**: Using MySQL to store user details, issues, and audit logs.
- **Responsive Design**: Ensuring the application adapts to different screen sizes using Tailwind CSS.

1.6 Significance of the Project

By implementing this system, institutions can:

- Reduce the time taken to resolve infrastructure problems.
- Provide transparency to students and staff regarding the progress of their complaints.
- Maintain digital records that support decision-making, budgeting, and long-term maintenance planning.
- Enhance the overall campus experience, improving satisfaction among stakeholders.
- Establish accountability by tracking actions taken on each complaint.

1.7 Technological Context

The application uses:

- **Backend**: Python Flask framework for handling routes, business logic, and database operations.
- **Frontend**: HTML templates styled with Tailwind CSS for a modern, responsive design; JavaScript and Chart.js for interactivity and analytics.
- **Database**: MySQL for secure, persistent data storage.
- **Version Control**: GitHub repository for collaborative development and version tracking.

1.8 Anticipated Outcomes

Once deployed, **Fix My Campus** will:

- Provide a single point of contact for all campus-related infrastructure issues.
- Reduce miscommunication between students and administration.
- Offer administrators data-driven insights into recurring problems.
- Act as a foundation for further enhancements, such as mobile apps, notification systems, and AI-driven issue categorization.

CHAPTER 2

ABOUT THE COMPANY

Scontinent Technologies Pvt Ltd

2.1 Overview & Identity

- **Name:** Scontinent Technologies Private Limited
- **Incorporated:** 5 August 2021
- **Type:** Private Limited Company (non-government), limited by shares
- **Headquarters:** Bengaluru (Bangalore North), Karnataka, India
- **CIN (Corporate Identification Number):** U72900KA2021PTC150409

2.2 Mission, Vision & Values

From its public site, Scontinent describes its vision and mission roughly as:

- To be a **global leader in socio-educational technology**, building inclusive bridges between learning and livelihood.
- To **empower students and professionals** by creating a collaborative platform for skill development, knowledge exchange, and career advancement.

Values and focus areas include:

- Innovation, personalisation, intelligence, analytics, optimisation.
- Providing certified internships, career-ready courses, verified credentials, progress tracking.

2.3 Key Products and Services

Scontinent offers several EdTech-oriented products and services:

- **SamAI App** – an AI-powered exam preparation app aimed at helping aspirants for exams like NEET, CET, JEE, UPSC, KPSC to prepare via mock tests, personalised suggestions, analytics, etc.
- **Internship & Career Development Courses** – programs aimed at helping students gain industry-relevant skills and real world experience.
- **AI Resume Builder** – tool to help users create effective resumes/cover letters, likely by leveraging AI to tailor content to roles.

- **Software / AI-Application Development Services** – custom software development, AI application development, networking & collaboration, project progress tracking, certified solutions.

2.4 Company Structure & Financials

- **Authorized Capital:** ₹10,00,000 (10 Lakhs)
- **Paid-Up Capital:** ₹9,00,000 (9 Lakhs)
- **Directors:**
  - Kataveeranahalli Hanumantharayappa Vinay
  - Santosh Ganapathi
  - Bandappa
  - Additional Directors added more recently: Ramachandrappa Chethana, Shweta R Hanchinmani
- **Industry:** Information & Communication / EdTech / Online Learning Platforms / Internet industry.
- **Company Status:** Active.

2.5 Achievements, Reach, and Reputation

- The company has built its reputation in the EdTech sector, especially for exam preparation (SamAI) and internships.
- Employee reviews: On AmbitionBox, it has a rating of about **4.7 / 5** based on a small sample of reviews. Key strengths cited: skill development, company culture, work-life balance.

2.6 Address & Contact

- **Address:** 32, Santosh Soudha, Near Bank Of India ATM, Vidyanagar Cross, Bettahalasur, Bangalorenorth, Karnataka, India 562157
- **Email:** office@scontinent.com; internship.sconti@gmail.com
- **Phone:** +91 9353548879, +91 8050590850

CHAPTER 3

LITERATURE SURVEY

3.1 Introduction

A literature survey is essential to understand existing systems, their limitations, and how new solutions can improve upon them. In the context of campus infrastructure issue management, institutions have traditionally relied on manual processes or ad-hoc digital methods. This section reviews the current state of complaint and issue-tracking systems, their use in educational environments, and the technologies used to implement them.

3.2 Existing Manual and Semi-Digital Systems

Most educational institutions still use manual reporting for maintenance issues. Students or staff either:

- Fill paper forms that are forwarded to maintenance staff.
- Call or email the facilities department.
- Report issues verbally to wardens or administrators.

  These processes are simple but suffer from:

- Delays in forwarding complaints.
- No standardized format for describing problems.
- Lack of a feedback loop to inform the complainant of progress.
- No structured data for administrators to analyse recurring issues or allocate budgets.

  Some campuses have introduced semi-digital systems, such as Google Forms or internal email ticketing. While these reduce paperwork, they are not integrated platforms; administrators still manually sort, assign, and track issues, and complainants rarely see real-time status updates.

3.3 Web-Based Complaint Management Systems

The past decade has seen the rise of web-based issue tracking systems similar to help desk or customer support platforms. Examples include:

- Help Desk/ITSM Tools (e.g., Freshdesk, Zendesk): These are general customer support

systems that can be customized for educational environments but are expensive and feature-heavy.

- Open-Source Ticketing Systems : These provide basic ticket creation, status tracking, and assignment features but require technical expertise to deploy and customize.

  While effective in commercial contexts, these systems often lack:

- Campus-specific modules (roll number identification, issue categories like classroom or hostel maintenance).
- Integration with existing student databases.
- User interfaces tailored for students and staff rather than IT personnel.

3.4 Academic Studies and Institutional Initiatives

Research papers and case studies highlight the need for transparent complaint mechanisms in educational institutions:

- Facility Management in Higher Education (various studies): Stress the importance of timely maintenance for student satisfaction and institutional reputation.
- Digital Campus Initiatives: Universities worldwide have rolled out mobile apps for attendance, fees, and notices, but only a few incorporate infrastructure complaint modules.
- E-Governance in Education: Shows that digitizing administrative workflows improves transparency, reduces corruption, and enhances user satisfaction.

  Despite these advances, there is a noticeable gap: few dedicated, lightweight, and low-cost solutions for campus infrastructure issue management exist, particularly for institutions in developing countries.

3.5 Technologies Used in Existing Systems

From the literature and available systems, several technology stacks emerge:

- Backend Frameworks: PHP (Laravel), Python (Django/Flask), Node.js (Express).
- Databases: MySQL, PostgreSQL, MongoDB for storing complaints and user data.
- Front-End: HTML/CSS with frameworks like Bootstrap or Tailwind CSS for responsive design.

- Analytics: Libraries such as Chart.js or Plotly to visualize data.
- Deployment: Cloud platforms or on-premise servers.

  Each of these stacks has pros and cons in terms of cost, scalability, and ease of use. For educational institutions with limited budgets, open-source frameworks like Flask and MySQL offer a practical path.

3.6 Gaps Identified in the Literature

Based on the review of existing tools, the following gaps are identified:

1. Lack of Unified Platforms: Existing solutions are fragmented; most focus either on ticketing or analytics but not both in a campus context.
2. No Student-Friendly Interfaces: Many systems are designed for IT professionals rather than students or faculty.
3. Limited Data Visualization: Administrators need dashboards to quickly identify trends and recurring issues.
4. Insufficient Audit Trails: Few systems maintain detailed logs of actions taken, which are critical for accountability.
5. Cost and Complexity: Commercial solutions are often unaffordable for small and medium institutions.

3.7 Positioning of "Fix My Campus"

Fix My Campus is designed to fill these gaps by offering:

- A lightweight, open-source web application built with widely supported technologies (Python Flask, MySQL).
- Separate modules for students/staff (report and track) and administrators (manage, analyse, and audit).
- Responsive front-end built with Tailwind CSS for ease of access across devices.
- Interactive dashboards using Chart.js to visualize data by status, category, and time trends.

CHAPTER 4

REQUIREMENT SPECIFICATION

Software Requirements Specification (SRS)

4.1 Introduction

The Software Requirements Specification (SRS) outlines the complete set of requirements for the **Fix My Campus** web application. It defines the functional, non-functional, hardware, and software requirements necessary to develop, deploy, and maintain the system. This SRS serves as a reference for developers, testers, and stakeholders.

4.2 Purpose

The purpose of the **Fix My Campus** application is to provide a centralized platform for students and staff to report infrastructure and facility issues within an institution and for administrators to track, manage, and resolve these issues. The SRS defines what the system must do to fulfil this purpose.

4.3 Scope

- Provide secure login, signup, and password recovery for users.
- Allow students/staff to submit complaints about various issues (electricity, water, internet, furniture, cleanliness, etc.).
- Enable users to view the status of their submitted complaints.
- Provide an administrator module to view, filter, update, and delete issues.
- Maintain a digital audit log of actions performed by administrators.
- Provide visual analytics dashboards (issues by category, status, date trends).
- Ensure responsiveness across devices (desktop, tablet, mobile).
- Use open-source technologies for affordability and easy customization.

4.4 Overall Description

The system is a web-based application built with a Python Flask backend and a MySQL database. It uses HTML templates styled with Tailwind CSS and enhanced with JavaScript and Chart.js for interactivity and analytics. The system is deployed on a local server or cloud hosting platform and uses GitHub for version control.

4.5 User Characteristics

- **Students/Staff:** Non-technical users who log in, report issues, and view status updates. They expect a simple, intuitive interface.
- **Administrators:** Technical or semi-technical staff responsible for managing issues and user accounts. They require filtering options, dashboards, and audit logs.

4.6 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Server / Host Machine	Dual Core Processor, 4 GB RAM, 10 GB free disk space	Quad Core Processor, 8 GB RAM, SSD storage
Client Device	Any device with a modern browser (PC, tablet, smartphone)	Latest versions of Chrome/Firefox/Edge
Network	Internet connection with at least 1 Mbps bandwidth	Broadband connection for faster response

4.7 Software Requirements

Software	Details
Operating System	Windows 10/11, Linux (Ubuntu), or macOS
Backend Framework	Python Flask 2.3.3
Database	MySQL
Front-End	HTML5, Tailwind CSS, JavaScript, Chart.js
Python Libraries	Flask-MySQLdb, pandas

Web Server	Built-in Flask development server or production server like Gunicorn/Nginx
Version Control	Git and GitHub
Browser	Chrome, Firefox, Edge, or Safari

4.8 Functional Requirements

User Module

- **Login/Signup:** Users can register with first name, last name, email, mobile number, gender, and roll number. Users can log in using roll number and password.
- **Password Recovery:** Users can reset password by verifying roll number and mobile number.
- **Profile Management:** Users can view and update their personal details.
- **Report Issue:** Users can select an issue category, provide description and location, and submit.
- **Track Issues:** Users can view all previously reported issues and their status (Pending, In Progress, Resolved).
- **Change Password:** Users can change their password after authentication.

Administrator Module

- **Admin Login:** Secure login with predefined credentials.
- **Dashboard:** View total issues, status counts (Pending, In Progress, Resolved), and user statistics.
- **Manage Issues:** Filter issues by status, category, or roll number. Update status or delete issues.
- **Manage Users:** View all users, ban or unban users.
- **Audit Logs:** View actions taken by admin (updates, deletions, bans, etc.) with timestamps.
- **Analytics:** View charts of issues by status, category, and date trends.

System Module

- **Database:** Store user details, complaints, and audit logs securely.
- **Session Management:** Ensure secure login sessions for users and admins.
- **Responsive UI:** Adapt pages to various screen sizes.

4.9 Non-Functional Requirements

Requirement	Description
Performance	The system should handle concurrent users (e.g., 50–100 simultaneous requests).
Reliability	Data should be consistently stored and retrieved from the database without loss.
Availability	The application should be available 24/7 when hosted on a production server.
Scalability	The architecture should support adding new features like notifications or mobile apps.
Security	Passwords should be securely stored (hashing to be implemented in future), sessions protected against hijacking.
Usability	Simple and intuitive interface for non-technical users.
Maintainability	Clear code structure (separation of admin.py and app.py) for easier maintenance and upgrades.
Portability	Should run on different operating systems and browsers.
Auditability	All critical actions (status updates, deletions, bans) must be logged with timestamps.

4.11 Constraints

- Internet connection required for cloud deployment or external MySQL database.
- No mobile app currently — only web-based access.
- Admin credentials hardcoded initially (can be moved to database for production).

CHAPTER 5

PROJECT PLANNING

5.1 Project Management Methodology

For the “Fix My Campus” project, a Hybrid Agile approach has been adopted instead of the traditional Waterfall model. This modern methodology is flexible, adaptive, and highly suitable for academic projects, where timelines are limited and requirements may evolve during the course of development. Agile emphasizes iteration, feedback, and incremental delivery, which ensures continuous improvement and learning.

Reasons for adopting Agile-based planning are:

- Requirements in academic projects may change during testing or implementation.
- Encourages rapid prototyping and visible progress through weekly iterations.
- Provides flexibility to integrate advanced features such as analytics, mobile compatibility, and AI-based categorization.
- Supports individual project execution by dividing the work into smaller, manageable tasks (sprints).

5.2 Project Phases

The project development has been divided into six major phases, each with defined objectives and deliverables.

Phase 1: Requirement Gathering & Analysis

- Identify functional and non-functional requirements.
- Understand user workflows (students/staff for issue reporting and administrators for issue management).
- Define database entities and interactions between modules.

Phase 2: System & UI/UX Design

- Prepare architectural diagrams, ER models, and data flow diagrams.
- Design responsive and user-friendly UI using Tailwind CSS.
- Plan separate dashboards for administrators and end-users.

Phase 3: Implementation (Incremental Development)

- Backend: Develop Flask-based modules for authentication, issue reporting, tracking, and admin dashboard.
- Frontend: Build HTML templates styled with Tailwind CSS and integrate Chart.js for analytics.
- Database: Create schema in MySQL for users, issues, and audit logs.

Phase 4: Integration & Testing

- Perform unit testing for individual modules (login, signup, reporting).
- Conduct integration testing of backend, frontend, and database modules.
- Carry out usability and performance testing to ensure smooth user experience.

Phase 5: Deployment

- Deploy the system on a local Flask server for demonstration.
- Maintain a GitHub repository for version control and collaboration.
- Prepare for future deployment on cloud platforms such as AWS or Azure.

Phase 6: Review & Future Enhancements

- Collect feedback from students, staff, and faculty on usability and effectiveness.
- Identify improvement areas such as mobile application development, AI-driven issue categorization, and IoT-based automatic issue reporting.
- Plan for further scalability and multi-campus support.

5.3 Gantt Chart (Project Timeline – 14 Weeks)

The following Gantt Chart illustrates the project timeline across 14 weeks:

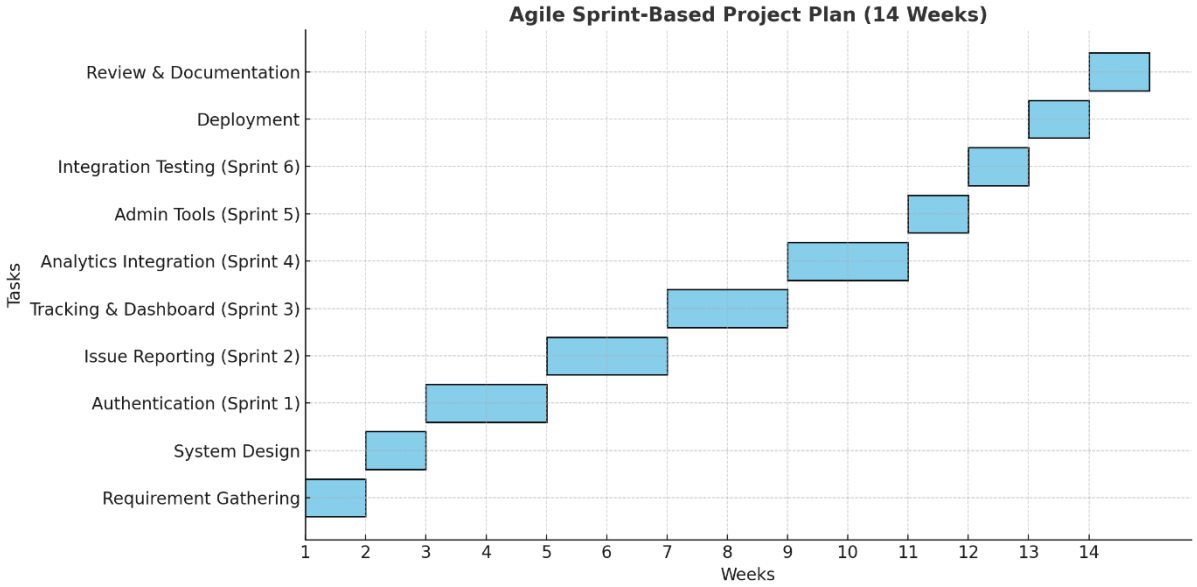


Figure 1: Gantt Chart illustrating the 14-week project timeline

CHAPTER 6

SYSTEM ANALYSIS

6.1 Introduction

The analysis phase identifies the system's needs, stakeholders, and workflows. It examines how users and administrators interact with the system, what data must be stored, and how processes flow from one component to another. For Fix My Campus, analysis ensures that all functional and non-functional requirements are clearly understood before design.

6.2 Stakeholder Analysis

Stakeholder	Role & Responsibilities
Students/Staff	Report issues, view status updates, update profiles.
Administrators	Manage all reported issues, update statuses, ban/unban users, view audit logs, and analyze trends.
Institution Management	Monitor dashboards and reports to allocate resources and improve infrastructure maintenance.

6.3 System Requirements Summary

- Secure login and signup.
- Issue reporting with category, description, and location.
- Status tracking (Pending, In Progress, Resolved).
- Admin dashboard for issue and user management.
- Visual analytics dashboards.
- Audit logs of admin actions.

6.4 Data Flow Analysis

User Module:

1. User logs in → enters roll number and password.
2. User reports an issue → form data stored in MySQL.
3. User views "My Issues" → retrieves their data from MySQL.

Admin Module:

1. Admin logs in → dashboard loads all issues from MySQL.
2. Admin filters or updates issue status → changes written back to MySQL.
3. Admin actions logged into audit_logs table.

6.5 Database Entities

- Users: Stores personal details, login credentials, and ban status.
- Issues: Stores complaints submitted by users with status and timestamps.
- Audit Logs: Stores records of admin actions (updates, deletions, bans).

6.6 Benefits of Analysis

- Clarifies workflows between students, staff, and administrators.
- Ensures all data points (roll number, category, status) are captured.
- Reduces risks of missing critical features during design and development.

CHAPTER 7

SYSTEM DESIGN

7.1 System Architecture

TheoreticalBackground:

A system architecture defines how the various components of the system interact with each other. For web applications, the client–server architecture is the most widely used model. In this architecture, the client (browser) sends requests to the server, which processes the requests, interacts with the database if needed, and returns the appropriate responses (HTML pages, JSON data, etc.).

The Fix My Campus application follows a tier client–server architecture:

1. **Client Layer (Presentation Layer):**
    - The end-users (students, staff, administrators) interact with the system through web browsers.
    - HTML templates styled with Tailwind CSS provide a clean, responsive design.
    - JavaScript adds interactivity (form validation, chart rendering using Chart.js).
2. **Application Layer (Flask Backend):**
    - Built with Python Flask, this layer handles routing, business logic, and communication between the user interface and the database.
    - Separate modules: app.py for students/staff, admin.py for administrators.
    - Implements authentication, issue reporting, status updates, analytics, and audit logging.

- **Main Application Server (app.py):**
    - Built with Python Flask framework
    - Handles all user-facing functionality
    - Serves HTML templates with dynamic content
    - Processes form submissions
    - Manages user sessions and authentication

- **Admin Application Server (admin.py):**
    - Separate Flask instance running on different port
    - Handles administrative functions
    - Provides enhanced security for privileged operations
    - Maintains audit logs of administrative actions
- **Responsibilities:**
    - Implement business logic and rules
    - Process HTTP requests
    - Manage user authentication and authorization
    - Interface with database
    - Generate dynamic content for templates

3. **Data Layer (MySQL Database):**
    - Stores persistent data such as user details, reported issues, and audit logs.
    - The Flask backend interacts with MySQL using Flask-MySQLdb.

- **MySQL Database:**
    - Relational database management system
    - Stores all persistent data
    - Maintains data integrity through constraints
    - Supports complex queries for reporting
- **Responsibilities:**
    - Data persistence
    - Transaction management
    - Data retrieval and manipulation
    - Maintaining data consistency

Data Flow

1. User Request: HTTP request from client browser
2. Routing: Flask routes the request to appropriate handler
3. Processing: Business logic executes, potentially querying database
4. Response Generation: Rendered HTML template sent back to client
5. Client Rendering: Browser renders HTML/CSS/JavaScript

Security Considerations

- Session-based authentication
- Password hashing (though current implementation uses plain text - needs improvement)
- SQL injection prevention through parameterized queries
- Role-based access control

Separate admin interface for privileged operations

Design Approach

The system uses a three-layer architecture:

- **Presentation Layer (Frontend):** HTML templates, Tailwind CSS and JavaScript. Provides user interface for students, and admin.
- **Application Layer (Backend):** Python Flask handles routing, form submissions, business logic, and communication with the database.
- **Data Layer (Database):** All persistent data, including users, issues, and audit logs, is stored in MySQL.

7.2 Architectural Design

User Browser <—-> Flask Application (app.py/admin.py) <—-> MySQL Database

- User actions in the browser (login, submit issue) go to Flask routes.
- Flask processes the request, interacts with the MySQL and the database, and returns HTML pages or JSON.

- The admin dashboard uses the same backend but different templates for admin-only pages.

7.3 Database Design

Entities and Attributes:

- Users: roll_no (PK), first_name, last_name, email, mob_num, gender, password, is_banned.
- Issues: issue_id (PK), roll_no (FK), issue_type, description, location, status, date_reported.
- Audit Logs: log_id (PK), action, details, timestamp.

Tables and Relationships:

Table Name	Key Attributes
Users	roll_no (PK), first_name, last_name, email, mob_num, gender, password, is_banned
Issues	issue_id (PK), roll_no (FK), issue_type, description, location, status, date_reported
Audit Logs	log_id (PK), action, details, timestamp

Database Schema Diagram

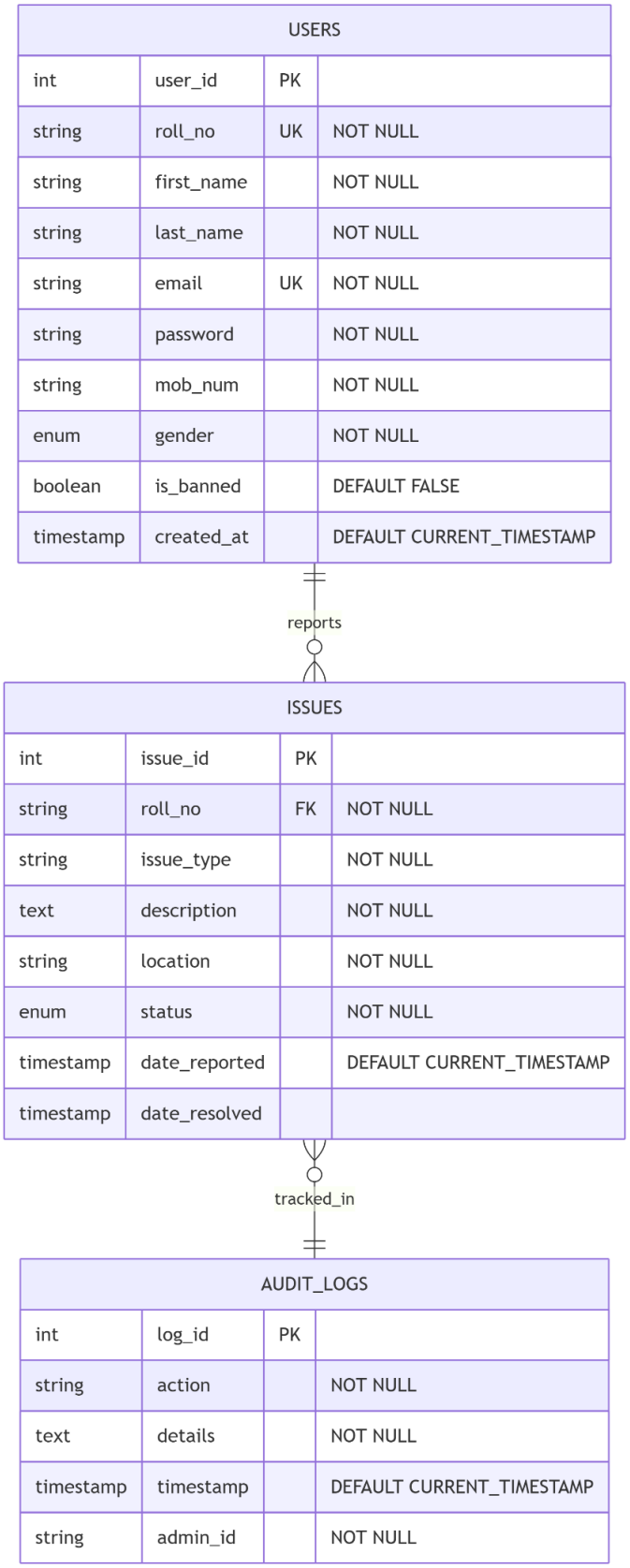


Figure 2: Database Schema Diagram

Data Flow Sequence Diagram

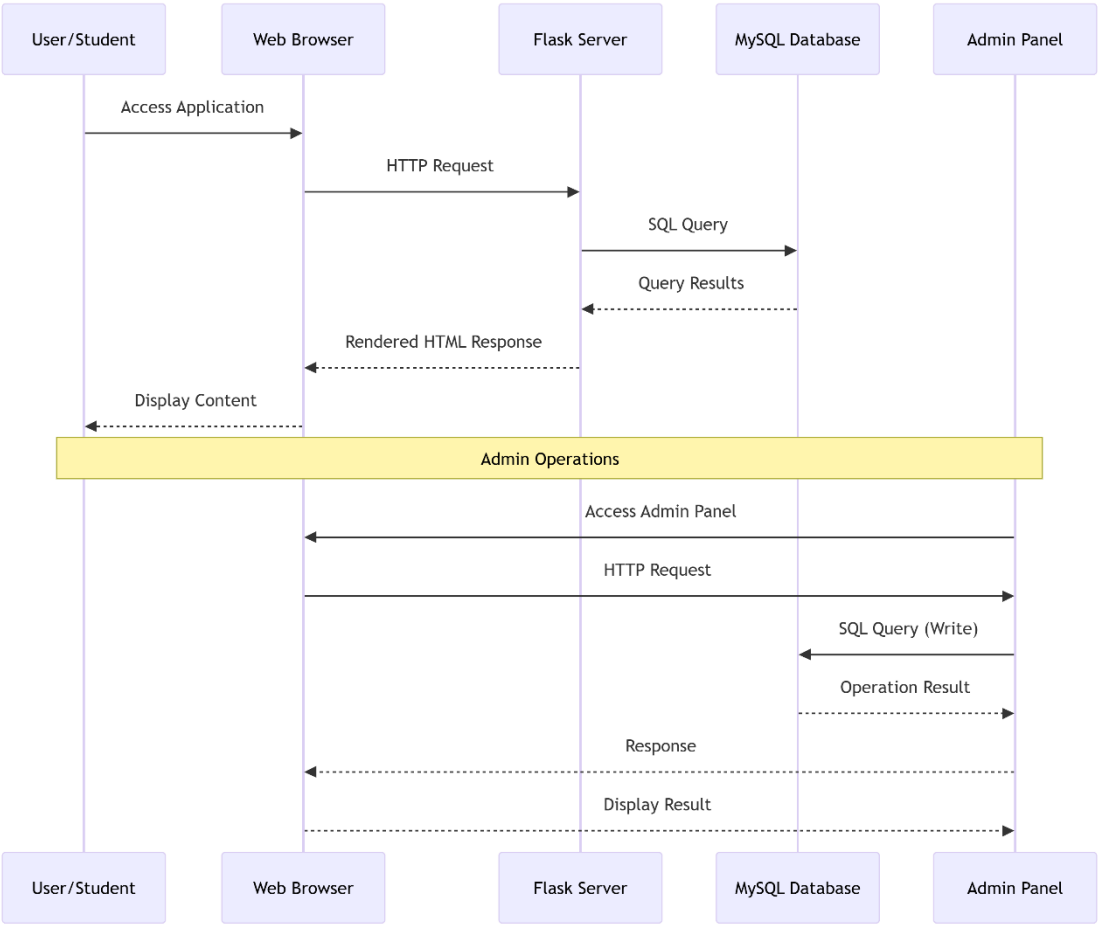


Figure 3: Data Flow Sequence Diagram

7.4 User Interface Design

- Login/Signup Pages: Simple forms, mobile-friendly using Tailwind CSS.
- User Dashboard: “Report Issue” form and “My Issues” list.
- Admin Dashboard: Table of all issues with filters, status counters, and charts.

7.5 Security Design

- Sessions used for authentication.
- Only admin can access admin routes.
- User passwords to be hashed in future updates.

7.6 Design Benefits

- Modularity: Separate files for user (app.py) and admin (admin.py).
- Reusability: HTML templates with base layout and components like navbar/footer.
- Scalability: Can add more modules (notifications, mobile API) without breaking architecture.

System User Flow & Architecture

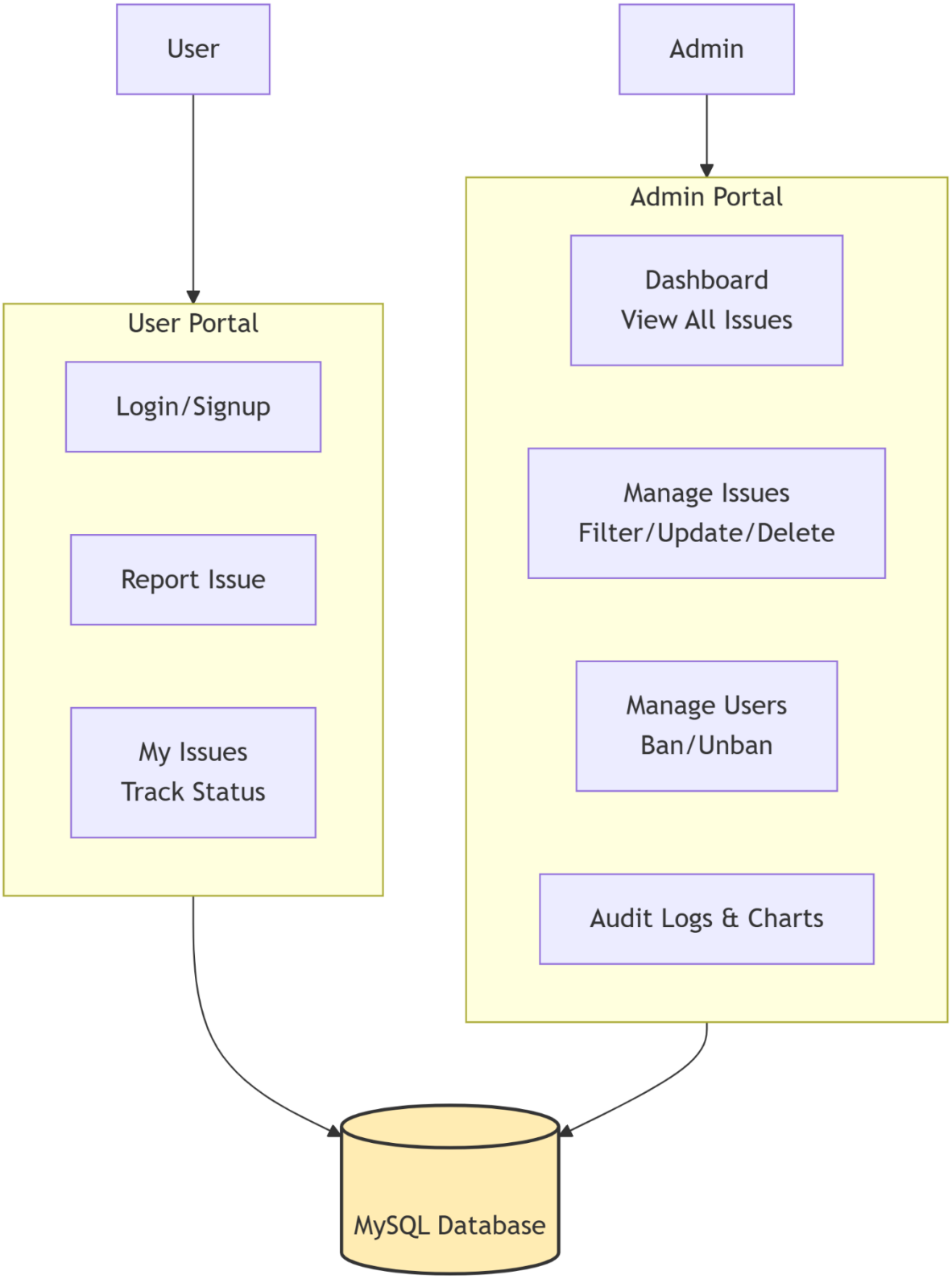


Figure 4: System User Flow & Architecture Diagram

System Architecture Diagram

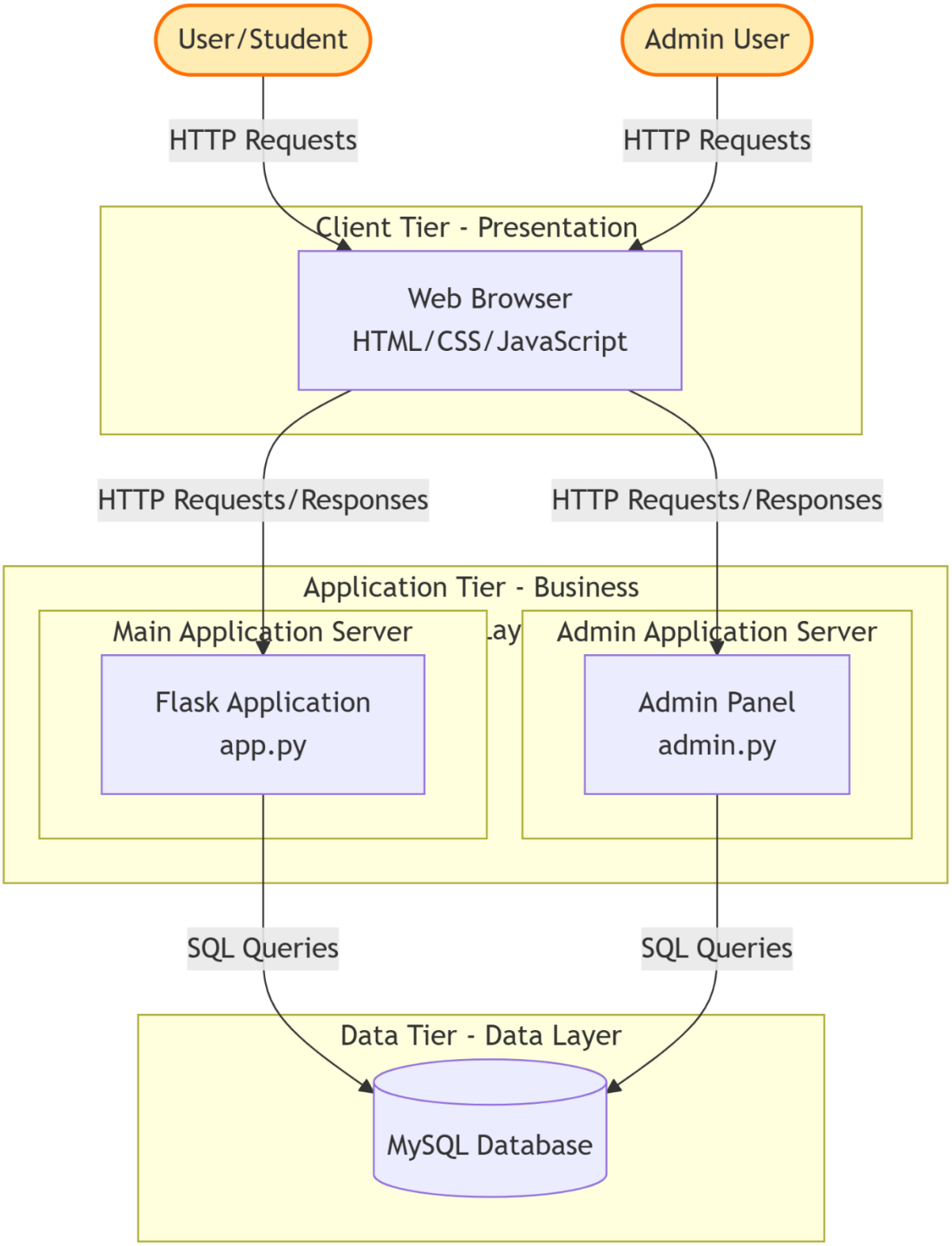


Figure 5: System Architecture Diagram

User Authentication Flow

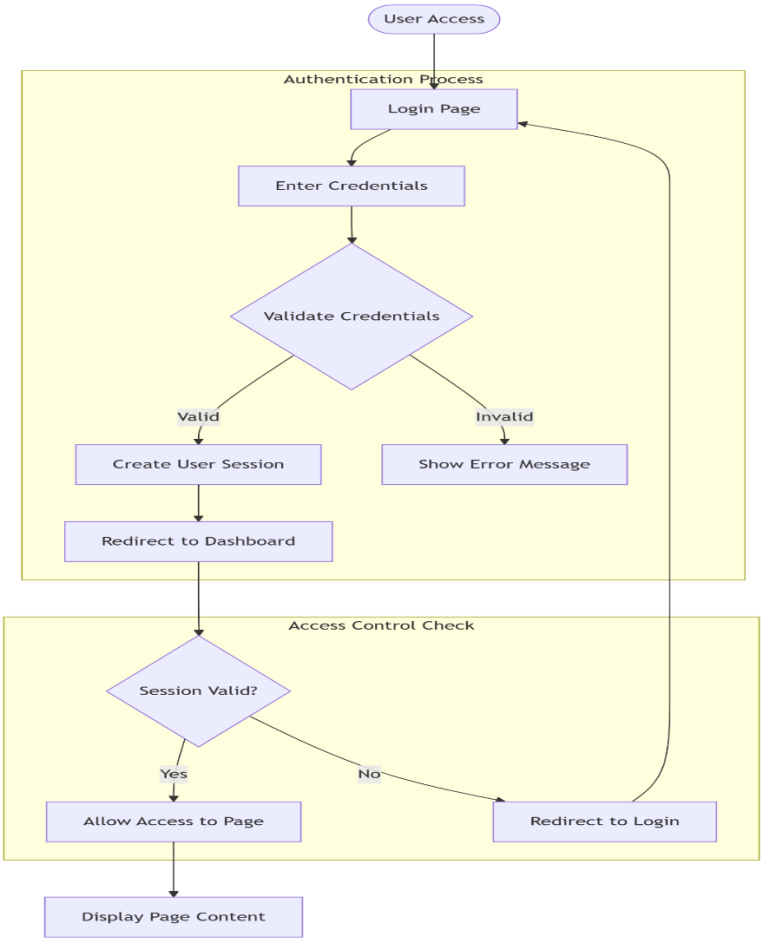


Figure 6: User Authentication Flow

Component Relationship Diagram

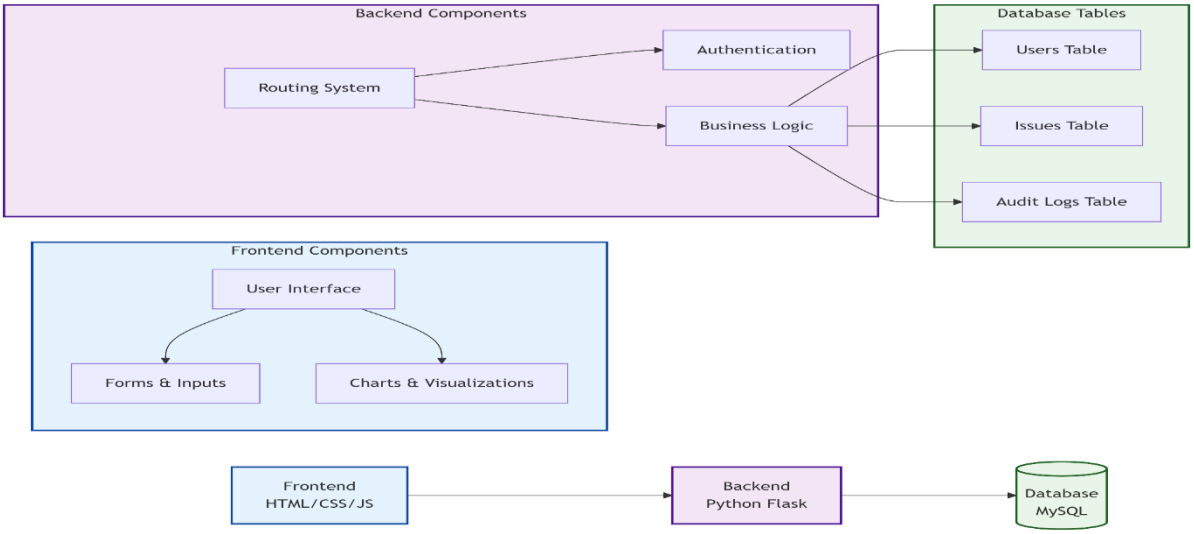


Figure 7: Component Relationship Diagram

3. UI/UX Design

Theoretical Background:

User Interface (UI) refers to the visual elements with which users interact, while User Experience (UX) refers to the overall usability and feel of the system. A good UI/UX ensures the system is easy to navigate, visually appealing, and accessible.

UI/UX Design in “Fix My Campus”:

- **Design Principles Used:**
  - Clarity: Minimalistic, straightforward forms for login, signup, and issue reporting.
  - Consistency: Same color palette and layout across pages using Tailwind CSS.
  - Responsiveness: Tailwind CSS utility classes ensure the application adapts to mobile, tablet, and desktop screens.
  - Feedback: Flash messages (success, danger, info) notify users about login success, errors, or updates.

- **Main Components:**
  - Login/Signup Pages: Clean forms with input validation.
  - Report Issue Page: Dropdown menu for categories, text area for description, location input.
  - My Issues Page: Table listing reported issues with status and dates.
  - Admin Dashboard:
    - Counters showing total issues by status.
    - Table of issues with filters.
    - Charts for analytics (status distribution, category distribution).
    - Audit log list showing admin actions.

Benefits of This UI/UX Design:

- Reduces learning curve for new users.
- Increases engagement and trust through a professional look.
- Mobile-friendly, so students can report issues on the go.

CHAPTER 8

TECHNOLOGIES STUDIED

Technology Studied

The Fix My Campus web application leverages a set of modern, open-source technologies to deliver a responsive, scalable, and maintainable system. The following section describes in detail the technologies studied and used for the backend, frontend, database, and version control/DevOps aspects of the project.

8.1 Backend Technologies

The backend is the core of the application that processes requests, applies business logic, interacts with the database, and sends responses to the client. For Fix My Campus, the following backend technologies were studied and implemented:

1.1 Python Flask

- **Overview:**
  Flask is a lightweight, open-source Web Server Gateway Interface (WSGI) web application framework written in Python. It provides the essentials to build web applications quickly without enforcing a specific project structure. This makes it ideal for small-to-medium applications where simplicity and flexibility are desired.
- **Key Features:**
  - Routing: Flask allows defining URL routes easily using decorators (@app.route).
  - Template Rendering: Integrates with the Jinja2 templating engine to render dynamic HTML pages.
  - Development Server: Built-in server for rapid testing and debugging.
  - Extensibility: Supports numerous extensions such as Flask-MySQLdb for database connectivity, Flask-Login for authentication, etc.
- **Use in Fix My Campus:**
  Flask handles all HTTP requests and responses. Separate modules (app.py for user functions and admin.py for administrator functions) were created to maintain

modularity. Examples include routes for login, signup, reporting issues, updating status, and displaying dashboards.

- **Benefits for the Project:**
  - Minimal setup, easy to learn.
  - Perfect for REST-style endpoints and template rendering.
  - Large community support and documentation.

1.2 Flask-MySQLdb

- **Overview**:
  Flask-MySQLdb is a Flask extension that simplifies connecting a Flask application to a MySQL database using the MySQLdb driver. It abstracts away connection boilerplate code, providing a convenient way to execute queries and manage transactions.
- **Key Features:**
  - Easy configuration using app.config dictionary for host, user, password, and database.
  - Returns results as dictionaries when using DictCursor for easier handling in Python.
  - Automatic connection pooling and cursor management.
- **Use in Fix My Campus:**
  This extension is used to:
  - Insert new user registrations into the users table.
  - Store new issues submitted by students.
  - Fetch data for dashboards and audit logs.
  - Update and delete records for administrative actions.
- **Benefits for the Project:**
  - Seamless integration with Flask.
  - Reduces code complexity.
  - Secure parameterized queries to prevent SQL injection.

8.2 Frontend Technologies

The frontend is responsible for the presentation layer — what users see and interact with in their browsers. For Fix My Campus, the goal was to provide a clean, intuitive, and responsive interface that works across devices.

2.1 HTML (Hypertext Markup Language)

- **Overview:**
  HTML is the standard markup language used to create the structure of web pages. It defines elements such as headings, paragraphs, forms, buttons, and links.
- Use in Fix My Campus:
  All pages — login, signup, report issue, dashboards — are created as HTML templates. Dynamic data is injected into these templates by Flask using Jinja2 syntax ({{ variable }}).
- Benefits for the Project:
  - Universally supported across browsers.
  - Works seamlessly with CSS and JavaScript.
  - Simple to write and maintain.

2.2 Tailwind CSS

- **Overview:**
  Tailwind CSS is a modern, utility-first CSS framework. Instead of pre-designed components, it provides low-level utility classes that can be combined to build any design directly in your HTML.
- **Key Features:**
  - Responsive Design: Built-in classes like sm:, md:, lg: for different screen sizes.
  - Customization: Highly configurable via tailwind.config.js.
  - Performance: Automatically purges unused styles in production.

- **Use in Fix My Campus:**
  Tailwind CSS styles all HTML templates to ensure:
  - A clean and modern look.
  - Responsive layouts for mobile and desktop.
  - Consistency in buttons, forms, tables, and dashboards.
- **Benefits for the Project:**
  - Faster UI development compared to writing CSS from scratch.
  - No need to override heavy default themes (like Bootstrap).
  - Maintains design consistency across pages.

2.3 JavaScript

- **Overview:**
  JavaScript is a client-side scripting language used to add interactivity to web pages. It runs directly in the browser and can dynamically manipulate HTML and CSS.
- **Use in Fix My Campus:**
  JavaScript is used to:
  - Validate forms before submission.
  - Enhance user experience with interactive components.
  - Render charts and graphs using Chart.js.
  - Filter data dynamically in dashboards.
- **Benefits for the Project:**
  - Enables real-time updates on pages without reloading.
  - Enhances usability with interactive features.
  - Works seamlessly with Flask-rendered templates.

2.4 Chart.js

- **Overview:**
  Chart.js is an open-source JavaScript library for creating responsive, interactive charts. It supports various chart types including bar, line, pie, and doughnut charts.

- **Key Features:**
  - Lightweight (only ~60kb).
  - Highly customizable with configuration options.
  - Responsive and mobile-friendly.
- **Use in Fix My Campus:**
  Used in the admin dashboard to display:
  - Issues by status (Pending, In Progress, Resolved).
  - Issues by category.
  - Issues by date trends.

8.3 Database Technologies

3.1 MySQL

- **Overview:**
  MySQL is a widely used open-source relational database management system (RDBMS). It stores data in tables and supports SQL for data manipulation.
- **Key Features:**
  - ACID-compliant transactions ensure data reliability.
  - High performance for read-heavy workloads.
  - Cross-platform support.
- **Use in Fix My Campus:**
  The MySQL database stores:
  - User details (roll number, name, email, password).
  - Issues submitted by users (type, description, location, status, timestamp).
  - Audit logs of admin actions (action, details, timestamp).
- **Benefits for the Project:**
  - Strong consistency and referential integrity.
  - Mature ecosystem and tools (MySQL Workbench).
  - Easy integration with Python via Flask-MySQLdb.

8.4 Version Control and DevOps Technologies

Version control and DevOps practices ensure the code is properly managed, versioned, and collaborative. For Fix My Campus, the following tools were studied and used:

4.1 Git

Overview:

Git is a distributed version control system used to track changes in source code during software development. It allows multiple developers to work on a project at the same time without overwriting each other's work.

Key Features:

- Supports both local and remote repositories.
- Allows branching and merging, making parallel development easier.
- Maintains a commit history for rollback, auditing, and tracking changes.

Usage in Fix My Campus:

- A local Git repository is initialized for the project.
- Changes are committed regularly after implementing features or fixing bugs.
- Separate branches are created for major features, such as the admin module, to organize development.

Benefits for the Project:

- Protects against accidental loss of code.
- Supports collaborative development if more team members are involved.
- Provides a clear and organized history of the project's progress and changes.

4.2 GitHub

Overview:

GitHub is a cloud-based hosting platform for Git repositories. It adds collaboration features such as pull requests, code reviews, and issue tracking.

Key Features:

- Web interface to browse repositories.
- Integration with CI/CD pipelines.

- Access control and project boards.

Use in Fix My Campus:

- Host the project repository at https://github.com/kmprasannakumar/Fix-My-Campus.git.
- Share code for review and backup.
- Track issues or feature requests using GitHub Issues.

Benefits for the Project:

- Acts as a single source of truth for the code.
- Enables future collaboration with other developers.
- Integrates with deployment workflows if needed.

Summary of Technologies

Category	Technologies
Backend	Python Flask, Flask-MySQLdb
Frontend	HTML, Tailwind CSS, JavaScript, Chart.js
Database	MySQL
Version Control/DevOps	Git, GitHub

\

CHAPTER 9

TECHNOLOGY IMPLEMENTATION

9.1 Introduction

This section explains how the technologies described in the previous section were applied in the actual development of the Fix My Campus application. It covers the implementation of the backend, frontend, database integration, and admin features. Each module is explained with its purpose, technology used, and code snippets. Screenshots are provided to illustrate the working system.

9.2 Environment Setup

- **Python and Flask Installation:** Installed Python 3.x and Flask (2.3.3) using pip install flask.
- **Database Setup:** Created a MySQL database named fmp with tables users, issues, and audit_logs.

9.3 Backend Implementation

9.3.1 Flask App Configuration

In app.py and admin.py, Flask is initialized and configured to connect to MySQL:

```
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'admin'
app.config['MYSQL_DB'] = 'fmp'
app.config['MYSQL_CURSORCLASS'] = 'DictCursor'
```

This allows the backend to execute SQL queries and return results as Python dictionaries.

9.3.2 User Authentication

- **Signup Route (/signup):** Collects user details and inserts them into the users table.
- **Login Route (/login):** Authenticates users by roll number and password. Banned users cannot log in.
- **Forgot Password Route (/forgot password):** Resets password after verifying roll number and mobile number.

Code :

```
@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        first_name = request.form['first_name']
        ...
        cur.execute(
          "INSERT INTO users (first_name, last_name, email, password, mob_num, gender,
roll_no) VALUES (%s,%s,%s,%s,%s,%s,%s)",
          (first_name, last_name, email, password, mob_num, gender, roll_no))
        mysql.connection.commit()
        flash('Registration successful!')
        return redirect(url_for('login'))
    return render_template('signup.html')
```

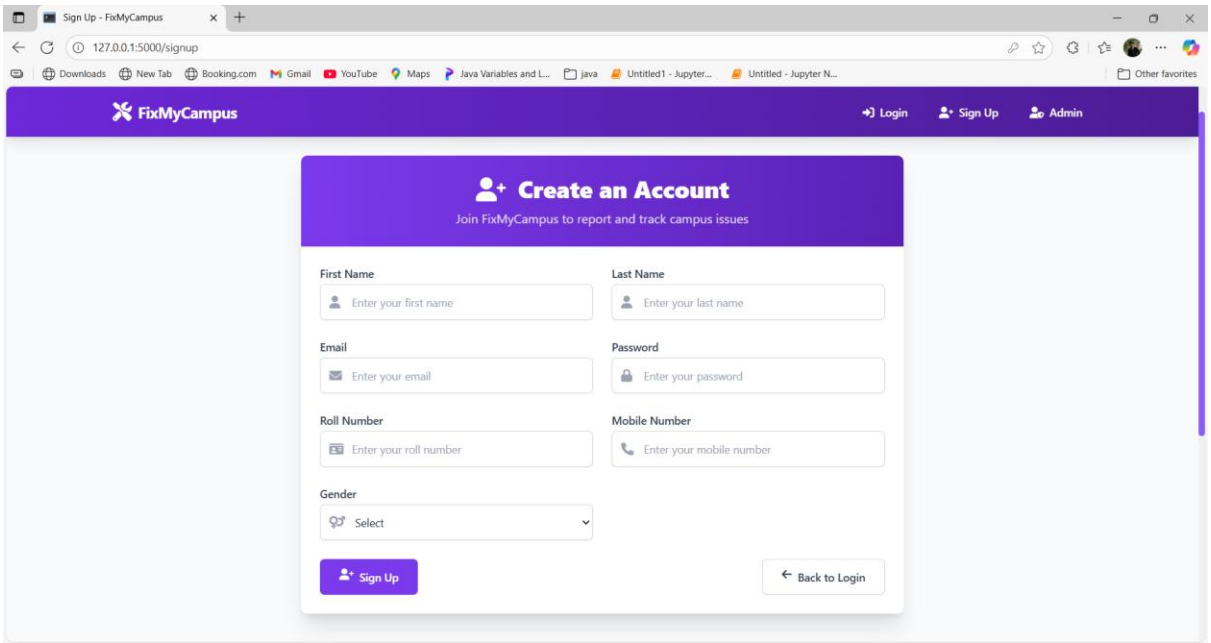


Figure 8: User Signup Page

9.3.3 Profile Management

Users can update their profile details. Changes are reflected in the users table.

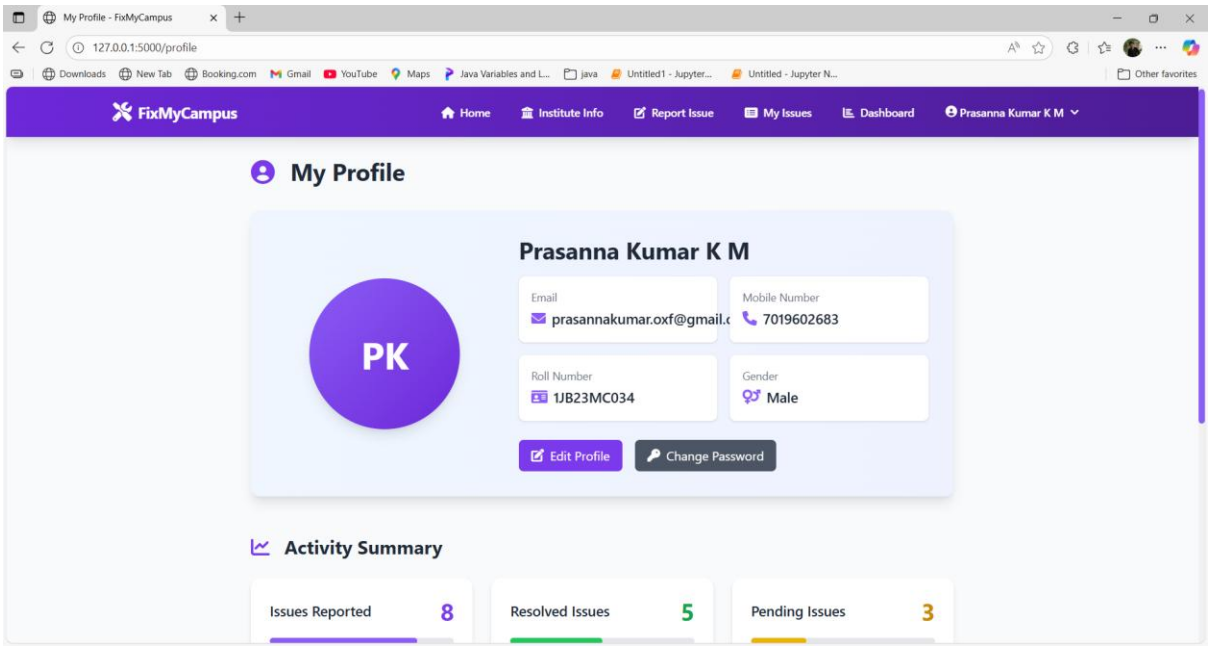


Figure 9: Profile page

9.3.4 Issue Reporting and Tracking

- **Report Issue (/report_issue):** Users select category, describe issue, and provide location. Status defaults to “Pending.”
- **My Issues (/my_issues):** Displays a table of issues submitted by the logged-in user.

Code:

```
@app.route('/report_issue', methods=['GET', 'POST'])
def report_issue():
    if request.method == 'POST':
        issue_type = request.form['issue_type']
        description = request.form['description']
        ...
        cur.execute('''
            INSERT INTO issues (roll_no, issue_type, description, location, status, date_reported)
            VALUES (%s, %s, %s, %s, %s, %s)
        ''', (session['roll_no'], issue_type, description, location, status, date_reported))
        mysql.connection.commit()
        flash('Issue reported successfully!')
```

```
        return redirect(url_for('my_issues'))
    return render_template('report_issue.html')
```

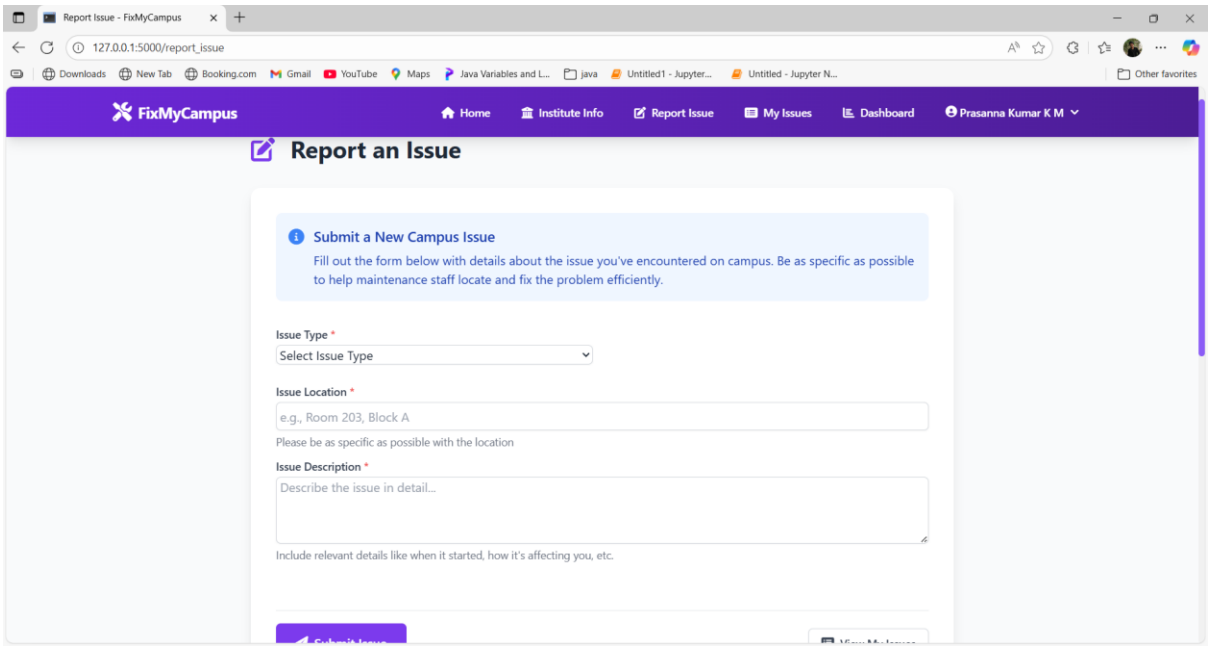


Figure 10: Report Issue form

9.4 Admin Module Implementation

9.4.1 Admin Authentication

Admin logs in with credentials admin@1234 / 123. Once logged in, a session variable is created to restrict admin pages.

9.4.2 Admin Dashboard

Displays:

- Total issues,
- Pending, In Progress, Resolved counts,
- List of users,
- Charts (status distribution, category distribution).

Code:

```
@app.route('/admin/dashboard')
def admin_dashboard():
    cur = mysql.connection.cursor()
    cur.execute('SELECT * FROM issues')
    issues = cur.fetchall()
```

return render_template('admin/dashboard.html', issues=issues, ...)

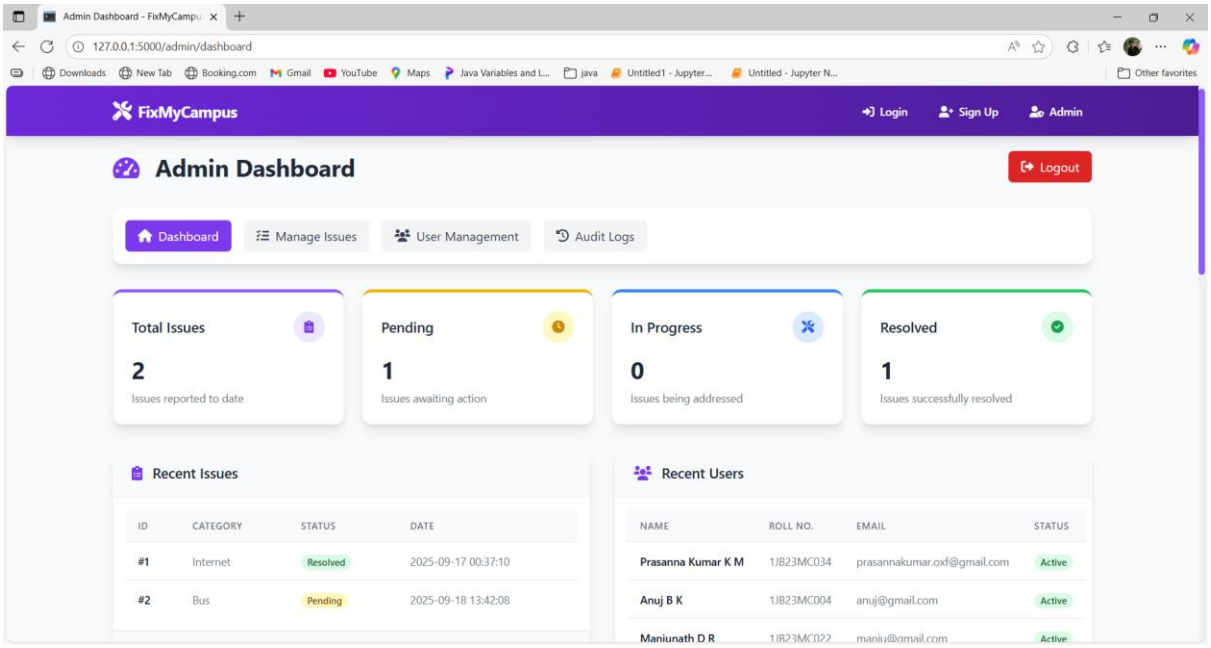


Figure 11: Admin

9.4.3 Manage Issues

Admin can:

- Filter issues by status, category, roll number.
- Update issue status (Pending → In Progress → Resolved).
- Delete issues.

Actions are logged in the audit_logs table.

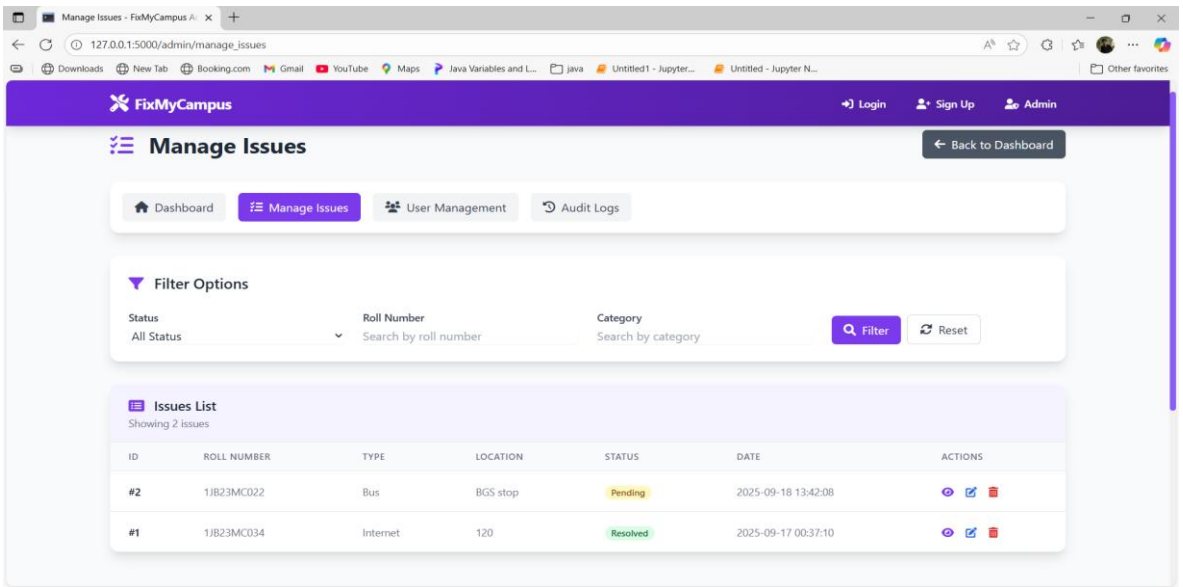


Figure 12: Manage issues

9.4.4 Manage Users

Admin can ban/unban users. A banned user cannot log in again.

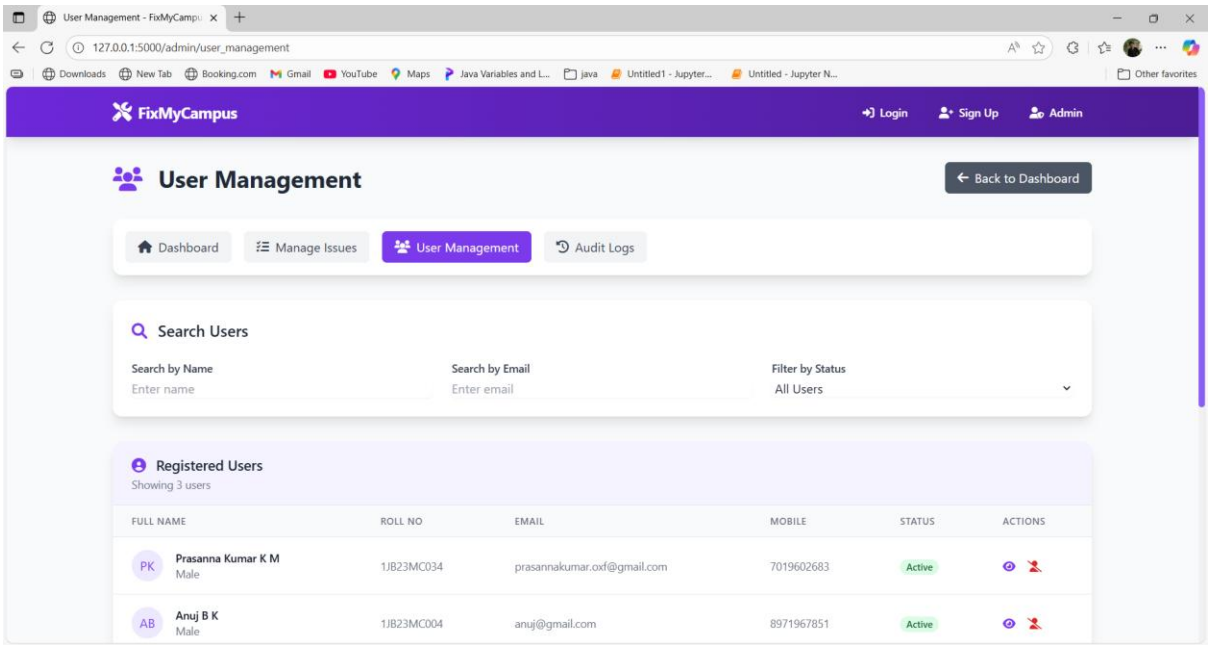


Figure 13: User management page

9.4.5 Audit Logs

All admin actions (status updates, deletions, bans) recorded in audit_logs with timestamp.

Screenshot: Audit log table in admin panel.

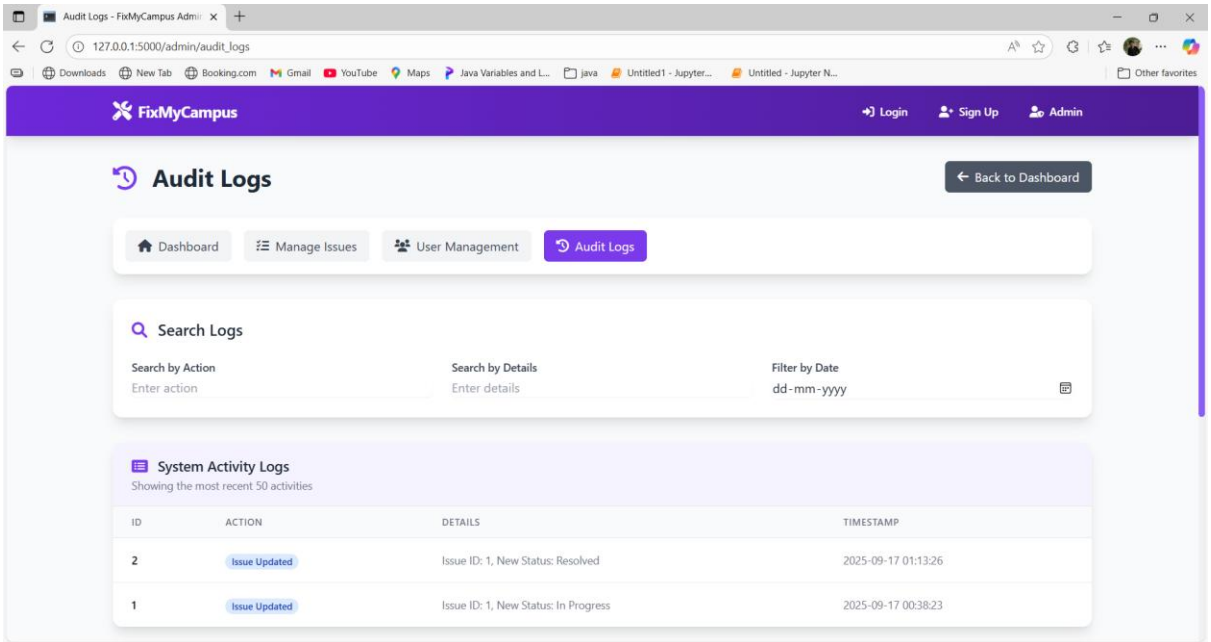


Figure 14: Audit log table in admin panel

9.5 Frontend Implementation

9.5.1 HTML Templates

Each page is a Jinja2 template. Layout components like navbar and footer are stored separately and included with {% include %} to reduce duplication.

Screenshot: Show part of HTML template code and the rendered page.

9.5.2 Tailwind CSS Styling

- Used classes like bg-gradient-to-br, rounded-2xl, p-10 for layout and styling.
- Ensures responsive design across devices.

Desktop View

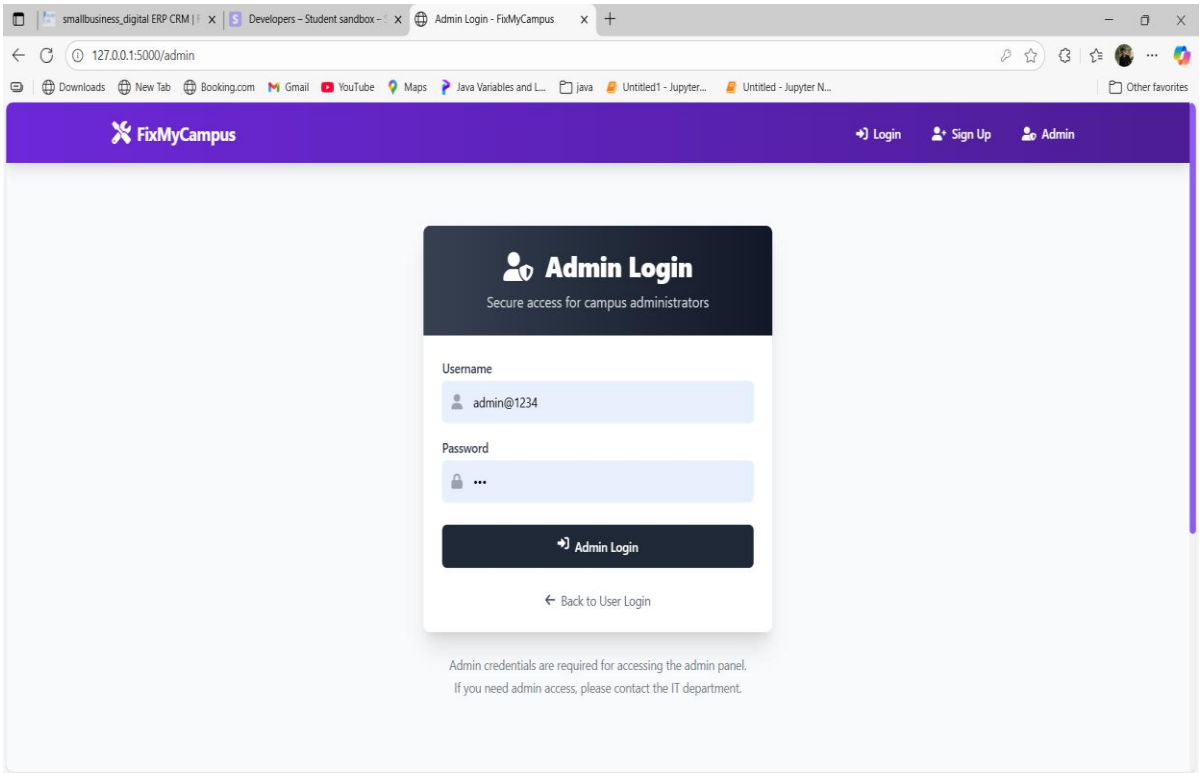


Figure 15: Fix My Campus Desktop View

Mobile View

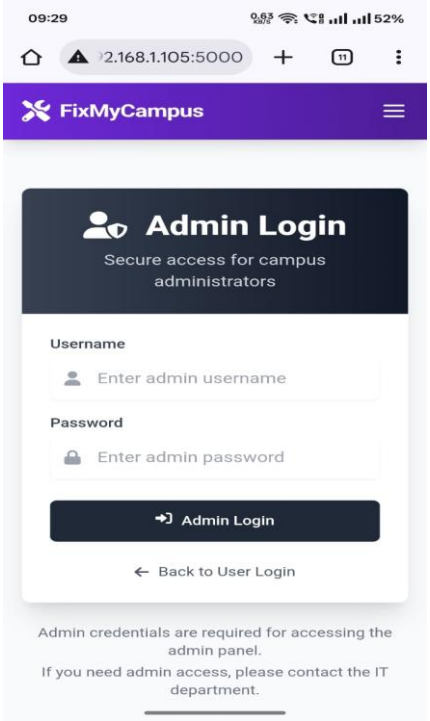


Figure 16: Fix My Campus Mobile View

9.5.3 JavaScript and Chart.js

- Client-side JavaScript validates forms before submission.
- Chart.js renders graphs showing issues by category and status.

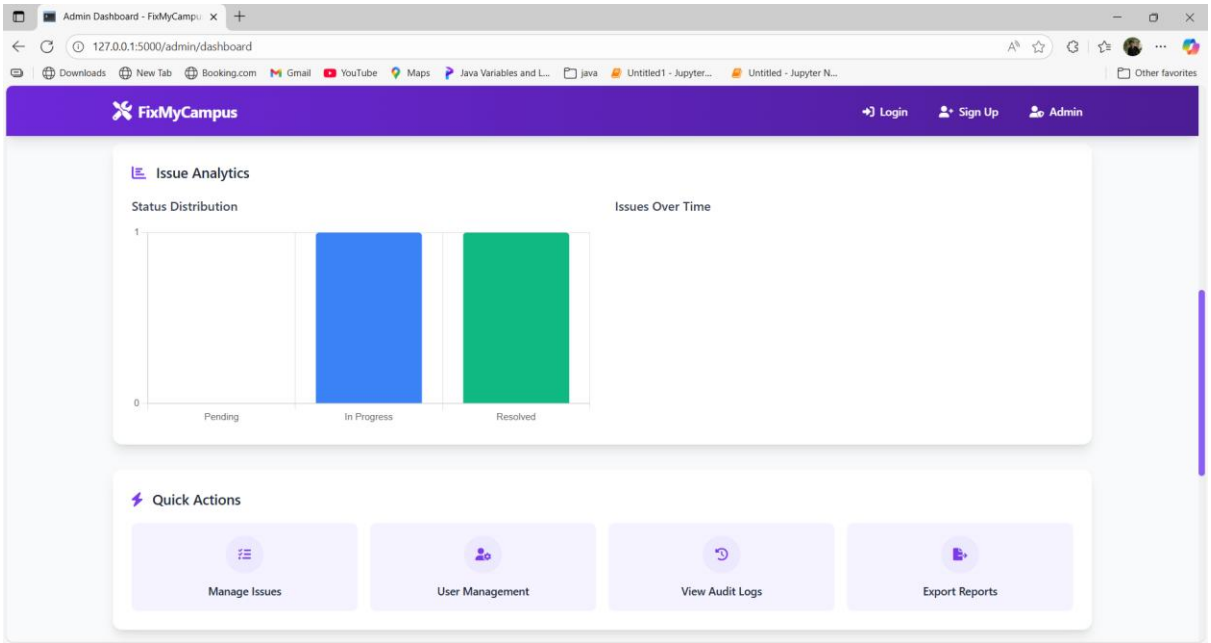


Figure 17: Admin dashboard showing bar charts

9.6 Database Implementation

- Created database fmp in MySQL.
- Created tables users, issues, and audit_logs.
- Used parameterized queries to prevent SQL injection.
- Verified data insertion by checking tables after actions.

Include SQL table creation statements:

```
CREATE TABLE users (
  roll_no VARCHAR(20) PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  email VARCHAR(100),
  password VARCHAR(100),
  mob_num VARCHAR(15),
  gender VARCHAR(10),
  is_banned TINYINT DEFAULT 0
);
```

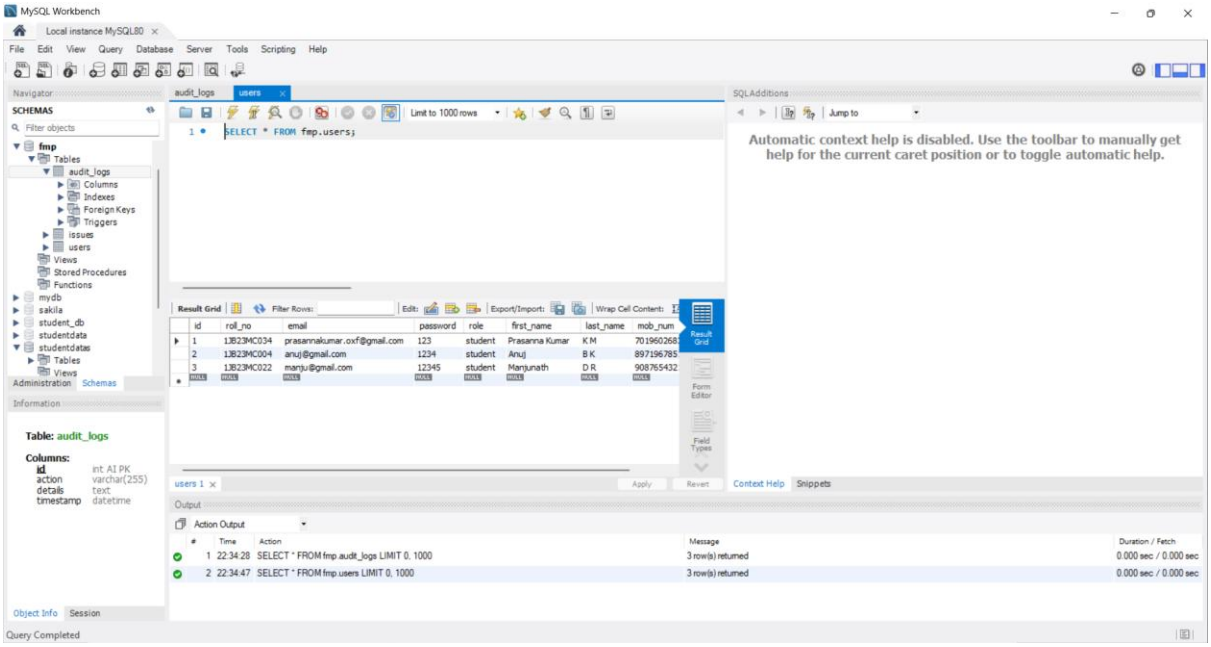


Figure 18: MySQL Workbench showing tables

9.7 Integration and Testing

- Integrated backend, frontend, and database gradually.
- Performed unit tests on routes, integration tests on workflows (login → report issue

→ admin updates status).

- Fixed bugs like incorrect status updates and missing flash messages.

9.8 Deployment and Version Control

- Hosted code on GitHub at https://github.com/kmprasannakumar/Fix-My-Campus.git.
- Used Git commits to track changes.
- Application can be run locally using:
- pip install -r requirements.txt
- python app.py
- python admin.py

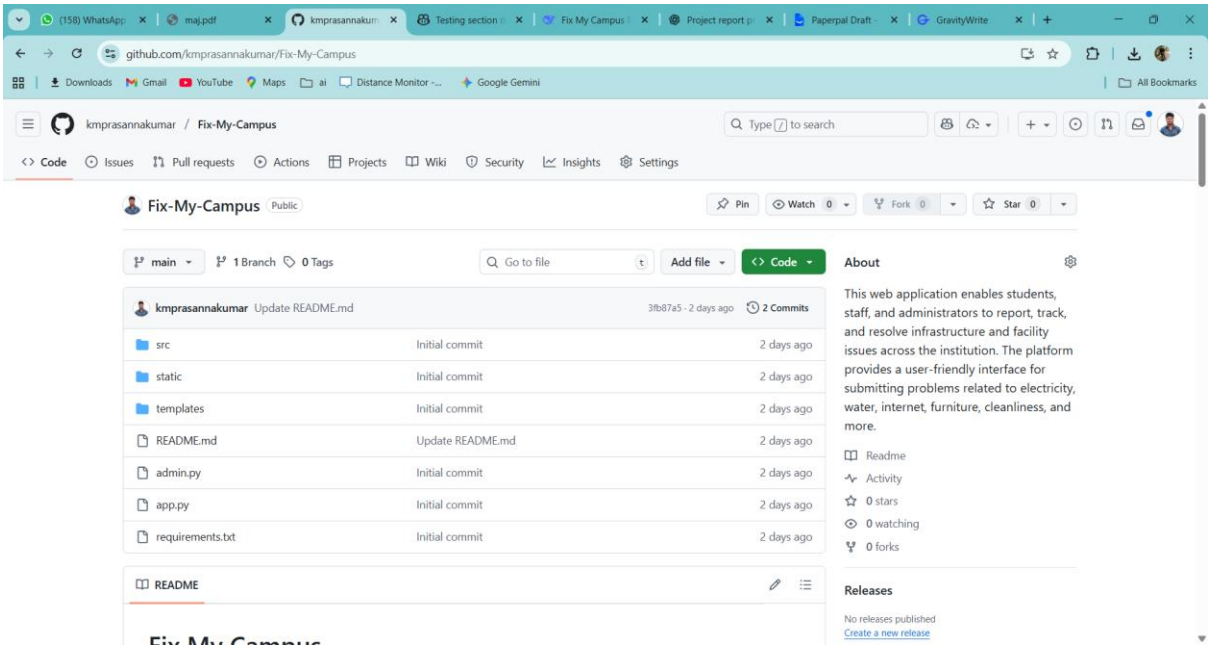


Figure 19: GitHub repository page

9.9 Screenshots

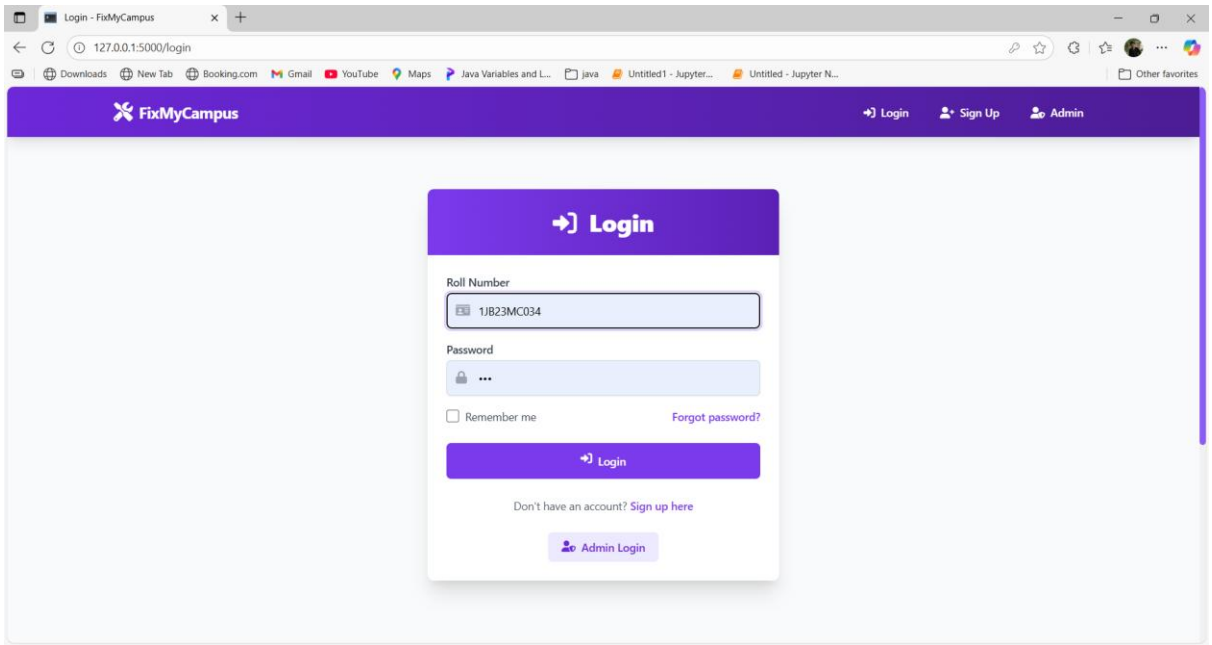


Figure 20: User Login Page

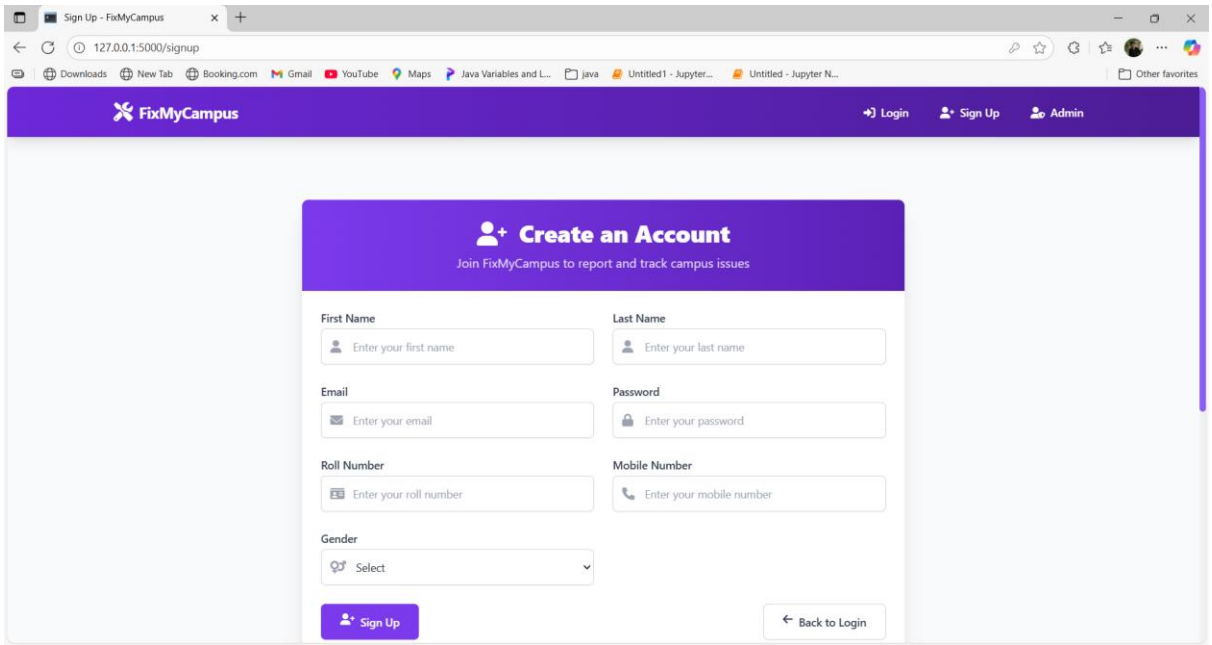


Figure 21: Signup Page

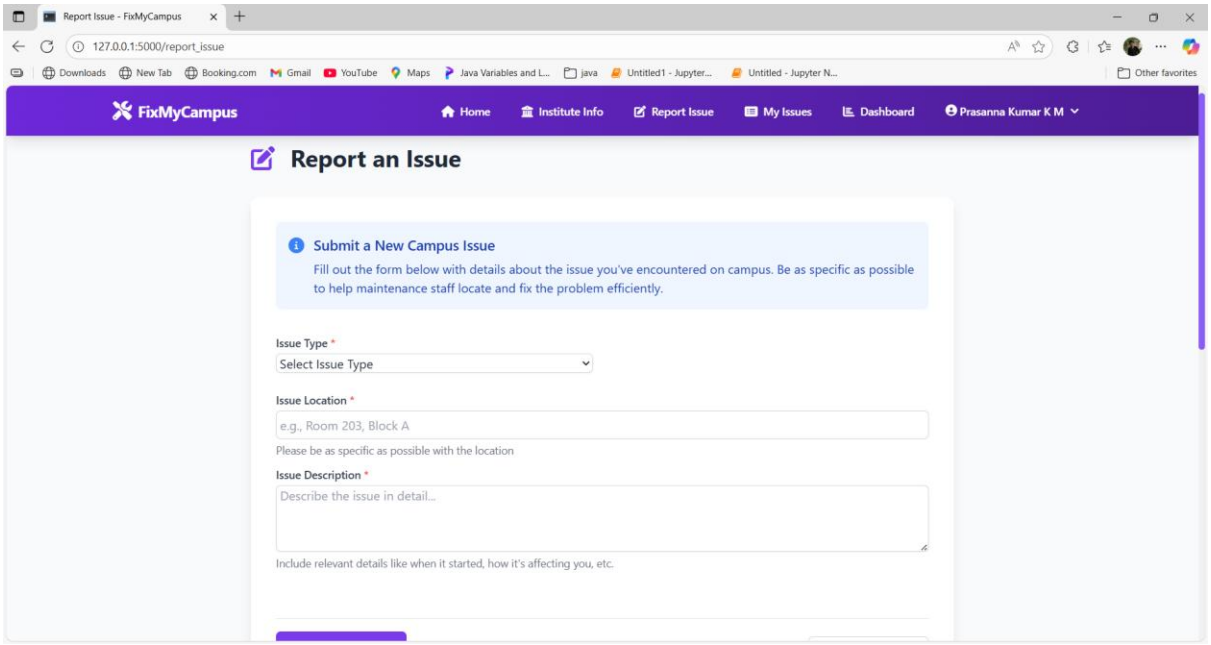


Figure 22: Report Issue Form

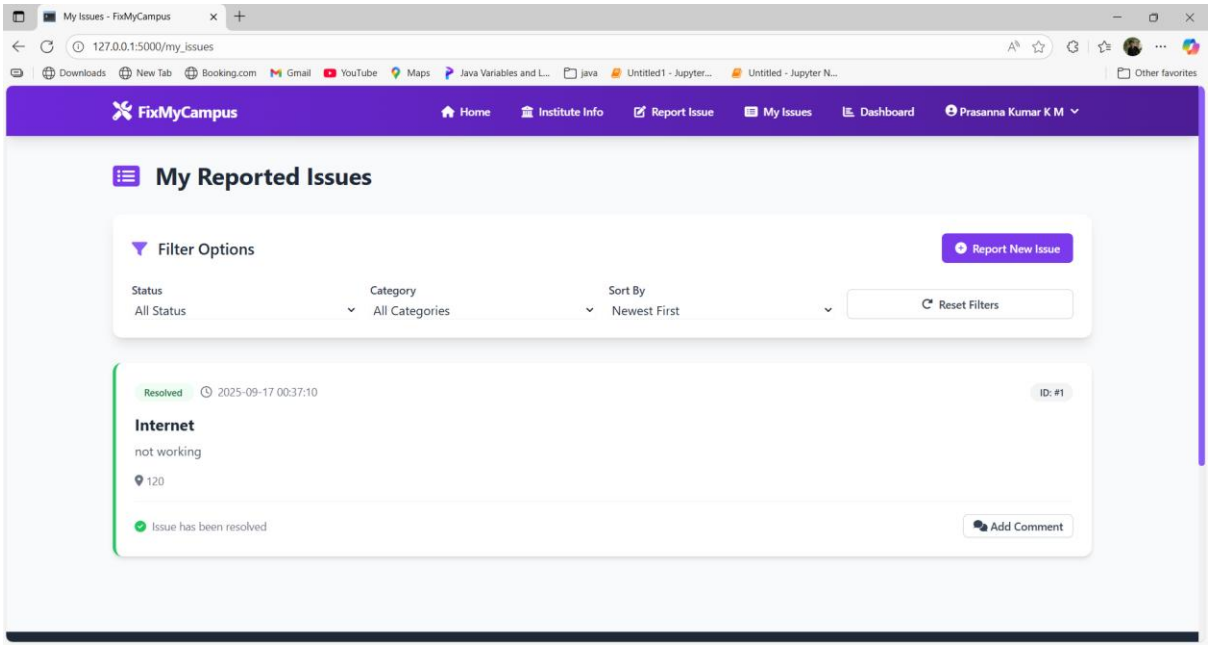


Figure 23: My Issues Page

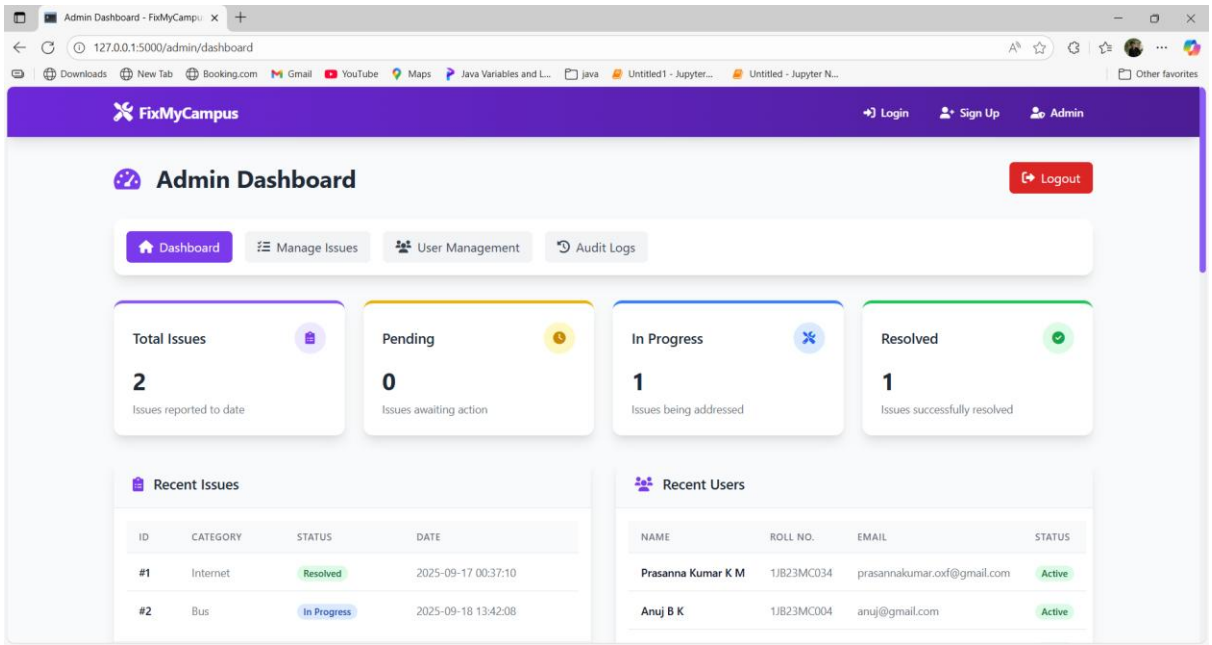


Figure 24: Admin Dashboard

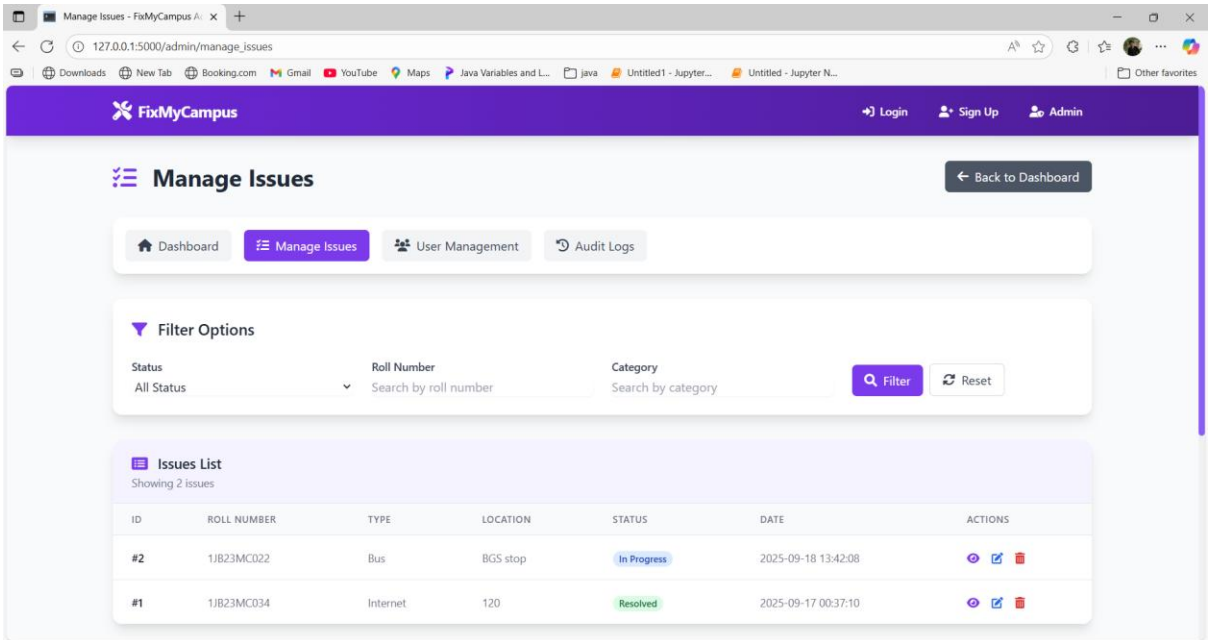


Figure 25: Manage Issues Page

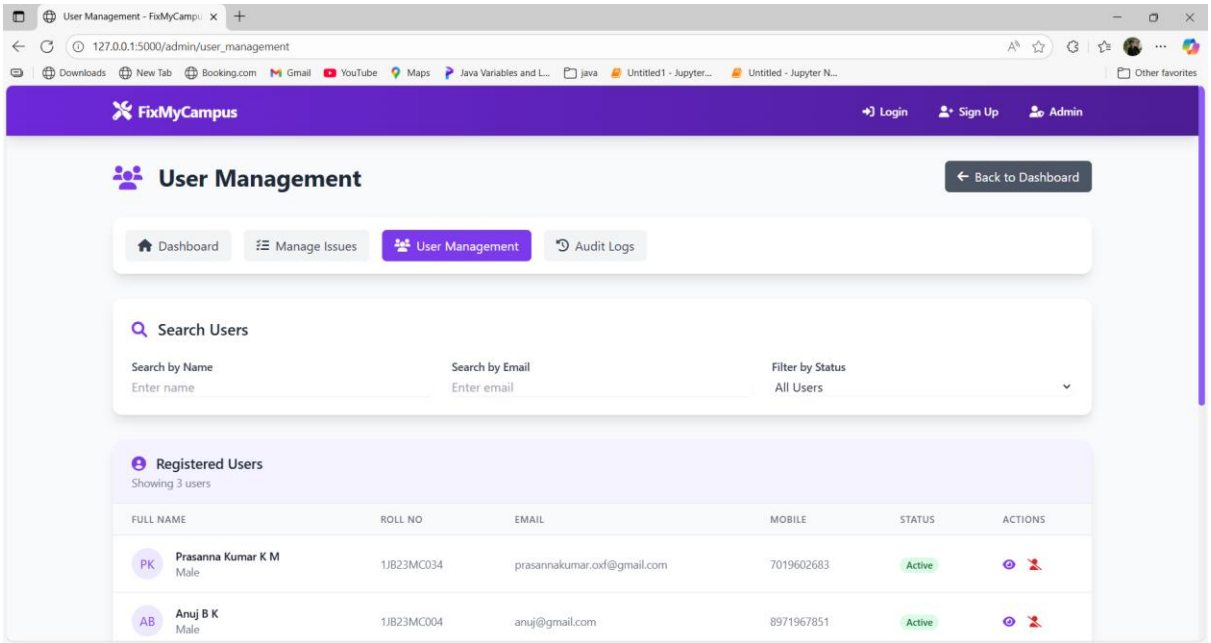


Figure 26: User Management Page

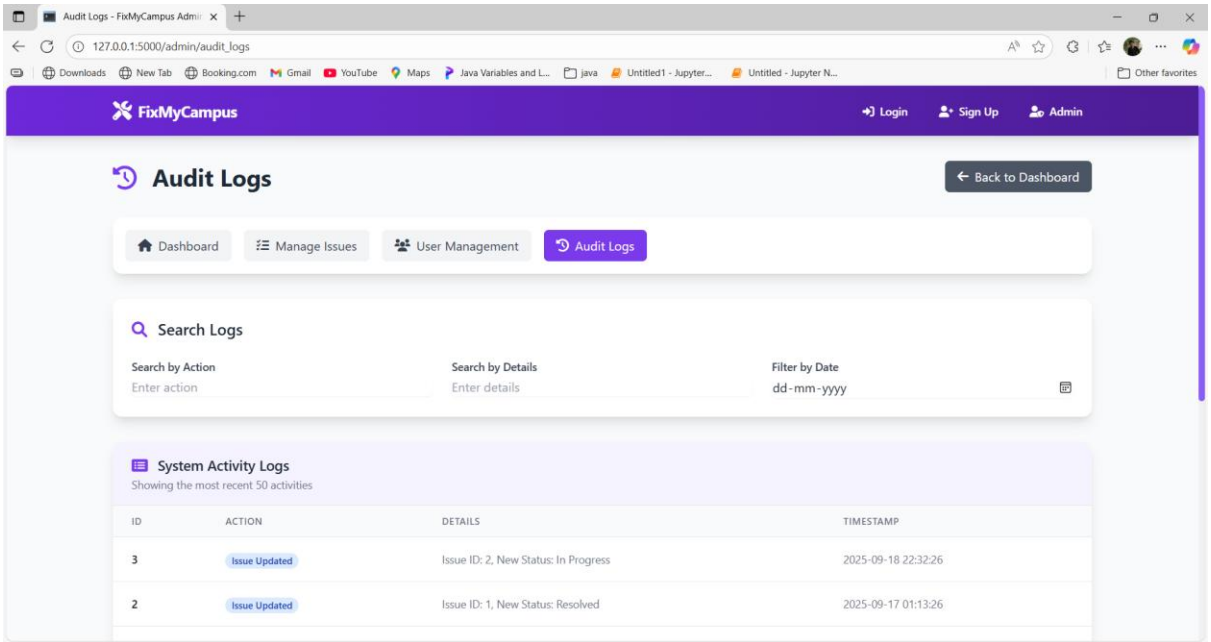


Figure 27: Audit Logs Page

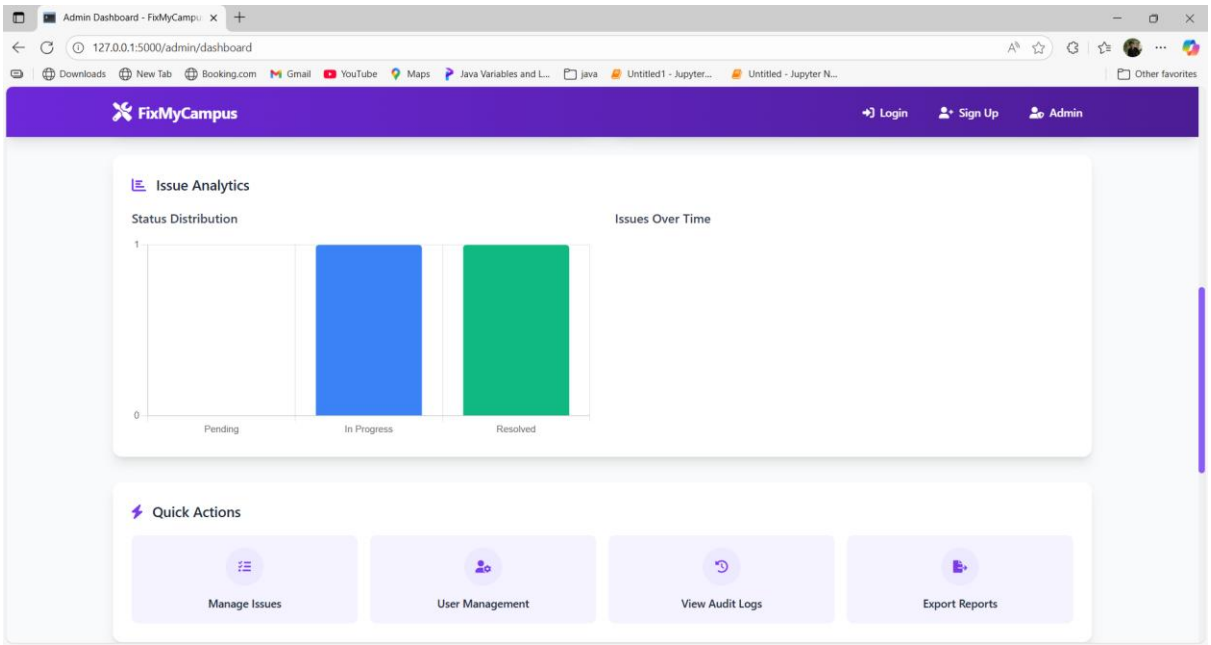


Figure 28: Charts showing issue trends

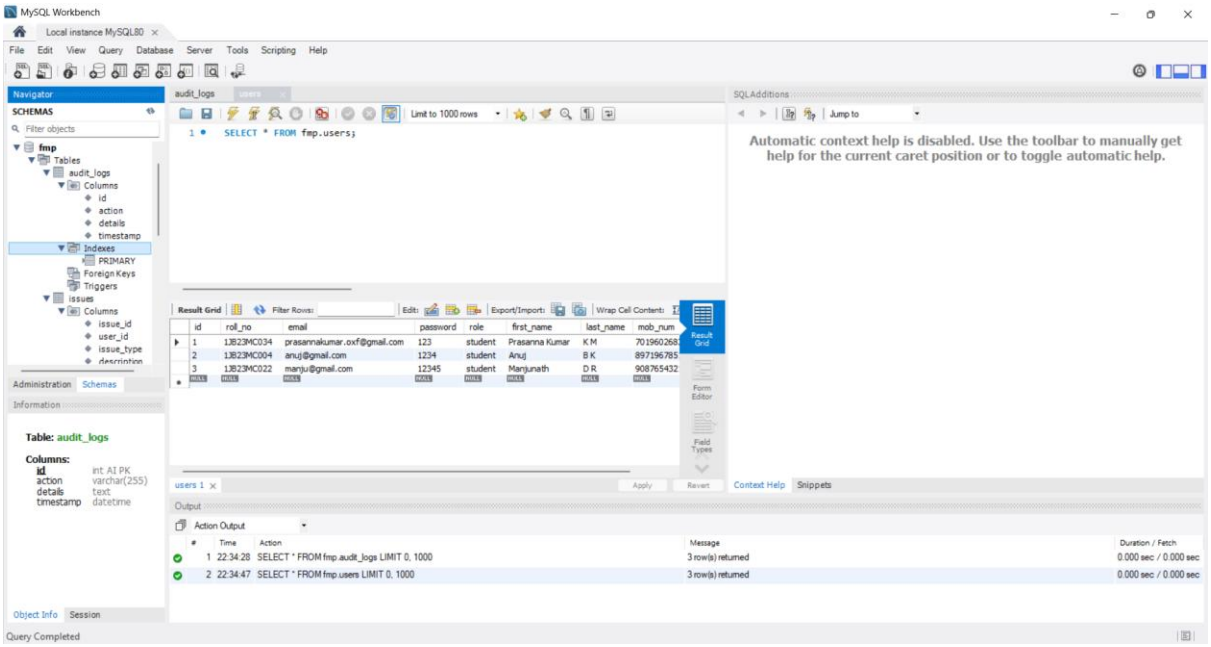


Figure 29: Database tables in MySQL Workbench

Code:

The **core functionality** of the "Fix-My-Campus" project is implemented using a modular architecture, combining front-end and back-end technologies for maintainability, scalability, and ease of development. The main components are described below:

App.py Main back-end application file.

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify
from flask_mysqldb import MySQL
import datetime

from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.secret_key = 'fixmycampus_secret_key'

# MySQL Configuration
# app.config['MYSQL_HOST'] = 'sql12.freesqldatabase.com'
# app.config['MYSQL_USER'] = 'sql12774989'
# app.config['MYSQL_PASSWORD'] = 'acxEkHFzcu'
# app.config['MYSQL_DB'] = 'sql12774989'
# app.config['MYSQL_CURSORCLASS'] = 'DictCursor'

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'admin'
app.config['MYSQL_DB'] = 'fmp'
app.config['MYSQL_CURSORCLASS'] = 'DictCursor'

mysql = MySQL(app)

# Authentication routes
@app.route('/')
def index():
```

```
    if 'logged_in' in session:
        return redirect(url_for('home'))
    return render_template('login.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        roll_no = request.form['roll_no']
        password = request.form['password']

        cur = mysql.connection.cursor()
        cur.execute("SELECT * FROM users WHERE roll_no = %s", [roll_no])
        user = cur.fetchone()

        if user and user['password'] == password:
            if user['is_banned']:
                flash('Your account has been banned. Please contact admin.', 'danger')
                return render_template('login.html')

            session['logged_in'] = True
            session['roll_no'] = user['roll_no']
            session['username'] = f"{user['first_name']} {user['last_name']}"
            flash('Login successful!', 'success')
            return redirect(url_for('home'))
        else:
            flash('Invalid credentials', 'danger')

    return render_template('login.html')

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        first_name = request.form['first_name']
        last_name = request.form['last_name']
```

```
        email = request.form['email']
        password = request.form['password']
        mob_num = request.form['mob_num']
        gender = request.form['gender']
        roll_no = request.form['roll_no']

        cur = mysql.connection.cursor()
        try:
            cur.execute(
                "INSERT INTO users (first_name, last_name, email, password, mob_num, gender,
roll_no) VALUES (%s, %s, %s, %s, %s, %s, %s)",
                (first_name, last_name, email, password, mob_num, gender, roll_no)
            )
            mysql.connection.commit()
            cur.close()
            flash('Registration successful! Please login.', 'success')
            return redirect(url_for('login'))
        except Exception as e:
            flash(f'Error: {str(e)}', 'danger')

    return render_template('signup.html')

@app.route('/logout')
def logout():
    session.clear()
    flash('You have been logged out', 'info')
    return redirect(url_for('login'))

@app.route('/forgot_password', methods=['GET', 'POST'])
def forgot_password():
    if request.method == 'POST':
        roll_no = request.form['roll_no']
        mob_num = request.form['mob_num']
        new_password = request.form['new_password']
```

```
        cur = mysql.connection.cursor()
        cur.execute("UPDATE users SET password = %s WHERE roll_no = %s AND
mob_num = %s",
                    [new_password, roll_no, mob_num])
        if cur.rowcount > 0:
            mysql.connection.commit()
            flash('Password reset successful!', 'success')
            return redirect(url_for('login'))
        else:
            flash('No matching user found with that Roll Number and Mobile Number.', 'danger')

    return render_template('forgot_password.html')

# User routes
@app.route('/home')
def home():
    if 'logged_in' not in session:
        return redirect(url_for('login'))
    return render_template('home.html', username=session['username'])

@app.route('/profile')
def profile():
    if 'logged_in' not in session:
        return redirect(url_for('login'))

    cur = mysql.connection.cursor()
    cur.execute("SELECT first_name, last_name, email, mob_num, gender, roll_no FROM
users WHERE roll_no = %s",
                [session['roll_no']])
    user = cur.fetchone()
    cur.close()

    return render_template('profile.html', user=user)
```

```
@app.route('/update_profile', methods=['POST'])
def update_profile():
    if 'logged_in' not in session:
        return redirect(url_for('login'))

    first_name = request.form['first_name']
    last_name = request.form['last_name']
    email = request.form['email']
    mob_num = request.form['mob_num']
    gender = request.form['gender']

    cur = mysql.connection.cursor()
    cur.execute('''
        UPDATE users
        SET first_name = %s, last_name = %s, email = %s, mob_num = %s, gender = %s
        WHERE roll_no = %s
    ''', (first_name, last_name, email, mob_num, gender, session['roll_no']))
    mysql.connection.commit()
    cur.close()

    session['username'] = f"{first_name} {last_name}"
    flash('Profile updated successfully!', 'success')
    return redirect(url_for('profile'))

@app.route('/change_password', methods=['GET', 'POST'])
def change_password():
    if 'logged_in' not in session:
        return redirect(url_for('login'))

    if request.method == 'POST':
        current_password = request.form['current_password']
        new_password = request.form['new_password']
        confirm_password = request.form['confirm_password']
```

```
        if new_password != confirm_password:
            flash('New passwords do not match', 'danger')
            return redirect(url_for('change_password'))

        cur = mysql.connection.cursor()
        cur.execute("SELECT password FROM users WHERE roll_no = %s",
[session['roll_no']])
        user = cur.fetchone()

        if user['password'] == current_password:
            cur.execute("UPDATE users SET password = %s WHERE roll_no = %s",
                    [new_password, session['roll_no']])
            mysql.connection.commit()
            cur.close()
            flash('Password updated successfully!', 'success')
            return redirect(url_for('profile'))
        else:
            flash('Current password is incorrect', 'danger')

    return render_template('change_password.html')

# Issue routes
@app.route('/report_issue', methods=['GET', 'POST'])
def report_issue():
    if 'logged_in' not in session:
        return redirect(url_for('login'))

    if request.method == 'POST':
        issue_type = request.form['issue_type']
        if issue_type == 'Other':
            issue_type = request.form['custom_issue_type']
        description = request.form['description']
        location = request.form['location']
```

```
        status = 'Pending'
        date_reported = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        cur = mysql.connection.cursor()
        cur.execute('''
            INSERT INTO issues (roll_no, issue_type, description, location, status,
date_reported)
            VALUES (%s, %s, %s, %s, %s, %s)
        ''', (session['roll_no'], issue_type, description, location, status, date_reported))
        mysql.connection.commit()
        cur.close()

        flash('Issue reported successfully!', 'success')
        return redirect(url_for('my_issues'))

    return render_template('report_issue.html')

@app.route('/my_issues')
def my_issues():
    if 'logged_in' not in session:
        return redirect(url_for('login'))

    cur = mysql.connection.cursor()
    cur.execute('''
        SELECT issue_id, issue_type, description, location, status, date_reported
        FROM issues
        WHERE roll_no = %s
        ORDER BY date_reported DESC
    ''', [session['roll_no']])
    issues = cur.fetchall()
    cur.close()

    return render_template('my_issues.html', issues=issues)
```

```
@app.route('/issue_dashboard')
def issue_dashboard():
    if 'logged_in' not in session:
        return redirect(url_for('login'))

    cur = mysql.connection.cursor()
    cur.execute('''
        SELECT roll_no, issue_id, issue_type, description, location, status, date_reported
        FROM issues
        ORDER BY date_reported DESC
    ''')
    issues = cur.fetchall()
    cur.close()

    # Prepare data for charts
    issue_count_by_status = {}
    issue_count_by_category = {}

    for issue in issues:
        # Count by status
        status = issue['status']
        if status in issue_count_by_status:
            issue_count_by_status[status] += 1
        else:
            issue_count_by_status[status] = 1

        # Count by category
        category = issue['issue_type']
        if category in issue_count_by_category:
            issue_count_by_category[category] += 1
        else:
            issue_count_by_category[category] = 1

    return render_template('issue_dashboard.html', issues=issues,
```

```
                    status_data=issue_count_by_status,
                    category_data=issue_count_by_category)

@app.route('/about_campus')
def about_campus():
    return render_template('about_campus.html')

@app.route('/help_support')
def help_support():
    return render_template('help_support.html')

@app.route('/about')
def about():
    return render_template('about.html')

# Admin routes
@app.route('/admin')
def admin_login():
    return render_template('admin/login.html')

@app.route('/admin/login', methods=['POST'])
def admin_login_post():
    username = request.form['username']
    password = request.form['password']

    if username == 'admin@1234' and password == '123':
        session['admin_logged_in'] = True
        return redirect(url_for('admin_dashboard'))
    else:
        flash('Invalid credentials', 'danger')
        return redirect(url_for('admin_login'))

@app.route('/admin/dashboard')
def admin_dashboard():
```

```
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    cur = mysql.connection.cursor()
    cur.execute('SELECT * FROM issues')
    issues = cur.fetchall()

    cur.execute('SELECT * FROM users')
    users = cur.fetchall()
    cur.close()

    pending_count = sum(1 for issue in issues if issue['status'] == 'Pending')
    in_progress_count = sum(1 for issue in issues if issue['status'] == 'In Progress')
    resolved_count = sum(1 for issue in issues if issue['status'] == 'Resolved')

    return render_template('admin/dashboard.html',
                           issues=issues,
                           users=users,
                           total_issues=len(issues),
                           pending_count=pending_count,
                           in_progress_count=in_progress_count,
                           resolved_count=resolved_count)

@app.route('/admin/manage_issues')
def admin_manage_issues():
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    status_filter = request.args.get('status', 'All')
    roll_no_filter = request.args.get('roll_no', '')
    category_filter = request.args.get('category', '')

    cur = mysql.connection.cursor()
```

```
    query = "SELECT * FROM issues WHERE 1=1"
    params = []

    if status_filter != 'All':
        query += " AND status = %s"
        params.append(status_filter)

    if roll_no_filter:
        query += " AND roll_no LIKE %s"
        params.append(f"%{roll_no_filter}%")

    if category_filter:
        query += " AND issue_type LIKE %s"
        params.append(f"%{category_filter}%")

    query += " ORDER BY date_reported DESC"

    cur.execute(query, params)
    issues = cur.fetchall()
    cur.close()

    return render_template('admin/manage_issues.html', issues=issues,
                           status_filter=status_filter,
                           roll_no_filter=roll_no_filter,
                           category_filter=category_filter)

@app.route('/admin/update_issue_status', methods=['POST'])
def admin_update_issue_status():
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    issue_id = request.form['issue_id']
    new_status = request.form['status']
```

```
    cur = mysql.connection.cursor()
    cur.execute("UPDATE issues SET status = %s WHERE issue_id = %s", [new_status,
issue_id])
    mysql.connection.commit()
    # Log the action
    action = "Issue Updated"
    details = f"Issue ID: {issue_id}, New Status: {new_status}"
    cur.execute(
        "INSERT INTO audit_logs (action, details, timestamp) VALUES (%s, %s, %s)",
        (action, details, datetime.datetime.now())
    )
    mysql.connection.commit()
    cur.close()
    flash('Issue status updated successfully!', 'success')
    return redirect(url_for('admin_manage_issues'))

@app.route('/admin/delete_issue/<int:issue_id>', methods=['POST'])
def admin_delete_issue(issue_id):
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    cur = mysql.connection.cursor()
    cur.execute("DELETE FROM issues WHERE issue_id = %s", [issue_id])
    mysql.connection.commit()
    # Log the action
    action = "Issue Deleted"
    details = f"Issue ID: {issue_id}"
    cur.execute(
        "INSERT INTO audit_logs (action, details, timestamp) VALUES (%s, %s, %s)",
        (action, details, datetime.datetime.now())
    )
    mysql.connection.commit()
    cur.close()
```

```
    flash('Issue deleted successfully!', 'success')
    return redirect(url_for('admin_manage_issues'))

@app.route('/admin/user_management')
def admin_user_management():
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM users")
    users = cur.fetchall()
    cur.close()

    return render_template('admin/user_management.html', users=users)

@app.route('/admin/ban_user/<string:roll_no>', methods=['POST'])
def admin_ban_user(roll_no):
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    cur = mysql.connection.cursor()
    cur.execute("UPDATE users SET is_banned = 1 WHERE roll_no = %s", [roll_no])
    mysql.connection.commit()

    # Log the action
    action = "User Banned"
    details = f"Roll No: {roll_no}"
    cur.execute(
        "INSERT INTO audit_logs (action, details, timestamp) VALUES (%s, %s, %s)",
        (action, details, datetime.datetime.now())
    )
    mysql.connection.commit()
    cur.close()
```

```
    flash('User banned successfully!', 'success')
    return redirect(url_for('admin_user_management'))

@app.route('/admin/unban_user/<string:roll_no>', methods=['POST'])
def admin_unban_user(roll_no):
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    cur = mysql.connection.cursor()
    cur.execute("UPDATE users SET is_banned = 0 WHERE roll_no = %s", [roll_no])
    mysql.connection.commit()

    # Log the action
    action = "User Unbanned"
    details = f"Roll No: {roll_no}"
    cur.execute(
        "INSERT INTO audit_logs (action, details, timestamp) VALUES (%s, %s, %s)",
        (action, details, datetime.datetime.now())
    )
    mysql.connection.commit()
    cur.close()

    flash('User unbanned successfully!', 'success')
    return redirect(url_for('admin_user_management'))

@app.route('/admin/audit_logs')
def admin_audit_logs():
    if 'admin_logged_in' not in session:
        return redirect(url_for('admin_login'))

    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM audit_logs ORDER BY timestamp DESC LIMIT 50")
    logs = cur.fetchall()
    cur.close()
```

```
    return render_template('admin/audit_logs.html', logs=logs)

@app.route('/admin/logout')
def admin_logout():
    session.pop('admin_logged_in', None)
    flash('Admin logged out', 'info')
    return redirect(url_for('admin_login'))

if __name__ == '__main__':
    app.run(host="0.0.0.0",port=5000,debug=True)
```

CHAPTER 10

TESTING

10.1 Unit Testing

Unit testing is carried out to check whether individual components of the system are functioning as expected. Each module is tested in isolation before integration with other parts of the system. In this project, unit tests were written for critical functions such as user authentication and issue submission. For example, the logic behind submitting a new issue was tested to confirm that data was stored correctly in the database, and session management was verified to ensure that only authenticated users could access restricted pages. The unit test framework in Python was mainly used for these checks.

10.2 Integration Testing

Integration testing ensures that the various modules of the system work together smoothly. After individual modules were tested, they were combined, and the flow of data between them was examined. For instance, the interaction between the issue reporting form on the frontend and the backend database was tested. Similarly, the communication between the admin dashboard and the issue-tracking module was validated. Tools like Postman and Selenium were useful in verifying these interactions.

10.3 Functional Testing

Functional testing was conducted to confirm that the system performs according to the specified requirements. Test cases were designed based on the functional requirements outlined in the SRS. For example, the end-to-end process of reporting an issue—from form submission to resolution by the administrator—was tested. Another scenario included checking whether users could filter issues based on categories such as “Cleanliness” or “Internet.” Manual testing and automated scripts were both applied to simulate these user interactions.

10.4 Usability Testing

Usability testing was performed to evaluate how user-friendly and intuitive the application is. Since the target users include students and staff who may not be technically skilled, the interface had to be simple and clear. Feedback was gathered from peers and faculty members on the ease of navigation, clarity of labels, and overall design of the interface. Suggestions from

these sessions were incorporated to improve the look and feel of the system.

10.5 Performance Testing

Performance testing assessed how the application behaves under different load conditions. This included measuring response times, throughput, and system stability when multiple users were accessing the system simultaneously. Tools like JMeter were used to simulate heavy traffic and concurrent issue submissions. The application was observed to respond within acceptable limits, even when subjected to higher-than-expected loads.

10.6 Security Testing

Security testing was critical to ensure that sensitive data such as login credentials and issue details remain protected. The system was checked for vulnerabilities such as SQL injection and cross-site scripting (XSS). Input validation was tested by deliberately entering malicious data into forms to confirm that it was handled safely. Data storage practices were also reviewed, ensuring that user details were stored securely in the database. Manual penetration testing techniques and security scanning tools were employed in this phase.

TEST CASES

1.User Authentication Module

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Test Type
UA-01	Signup with valid details	Open signup page → Enter valid data → Submit	First Name: John, Last Name: Doe, Email: john@example.com, Roll No: 22MCA001, Password: Pass@123, Mob: 9876543210, Gender: Male	Account created successfully, redirect to login page	Functional

UA-02	Signup with existing roll number	Enter already registered Roll No → Submit	Roll No: 22MCA001	Error message: “Roll Number already exists”	Functional
UA-03	Login with correct credentials	Enter Roll No & Password → Click login	Roll No: 22MCA001, Password: Pass@123	Successfully logged in, redirect to home page	Functional
UA-04	Login with wrong password	Enter valid Roll No but wrong password	Roll No: 22MCA001, Password: WrongPass	Display error “Invalid credentials”	Functional
UA-05	Forgot Password	Click “Forgot Password” → Enter Roll No & Mobile → Submit	Roll No: 22MCA001, Mob: 9876543210, New Password: Pass@456	Password reset successful, redirect to login	Functional / Security
UA-06	Session Timeout	Login and stay idle for 15 mins	—	User session expires, redirect to login page	Security / Usability

2. Issue Reporting Module

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Test Type
IR-01	Submit new issue	Go to Report Issue → Fill form → Submit	Category: Electricity, Description: "Room 101 light not working", Location: Block A	Issue added successfully, appears in "My Issues"	Functional
IR-02	Submit issue with empty fields	Leave mandatory fields empty → Submit	—	Display error "Please fill all required fields"	Functional / Validation
IR-03	Check issue ID generation	Submit multiple issues	—	Each issue gets unique Issue ID in DB	Functional
IR-04	Filter "My Issues" list	Go to "My Issues" → Apply filter (status or category)	Status: Pending	Only relevant issues shown	Functional / Integration

3. Issue Tracking Module

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Test Type
IT-01	Track pending issue	Open “My Issues”	—	Pending issues listed correctly	Functional
IT-02	Update issue status (Admin)	Admin logs in → Change status of issue from Pending → Resolved	Issue ID: 101	Status updated, visible to user	Functional / Integration
IT-03	View updated status	User logs in → My Issues	—	Status shows “Resolved”	Functional

4. User Interface & Usability

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Test Type
UI-01	Responsive layout	Open website on desktop, tablet, mobile	—	Layout adapts correctly	Usability
UI-02	Navigation links	Click all links in navbar/footer	—	Each link navigates to correct page	Usability
UI-03	Form field validation	Enter invalid email (without “@”) in signup	Email: johnexample.com	Display validation error	Functional / Validation

5. Security Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Test Type
SEC-01	SQL Injection	Enter ' OR 1=1 -- in login fields	Roll No / Password: ' OR 1=1 --	Login fails, no data breach	Security
SEC-02	XSS Attack	Enter <script>alert(1)</script> in issue description	—	Script sanitized, alert not executed	Security
SEC-03	Password storage (Backend)	Check database directly	—	Password stored as **hashed value** (future implementation)	Security

6. Performance Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Test Type
PERF-01	Load testing	Submit 50–100 issues simultaneously	—	System handles load without crash	Performance
PERF-02	Response time	Open dashboard and report issue page	—	Page loads within 2 seconds	Performance

CONCLUSION

The Fix-My-Campus project has successfully delivered a web-based application that addresses many of the difficulties faced by educational institutions in reporting, monitoring, and resolving infrastructure and facility-related issues. In most campuses, students, faculty, and staff regularly face challenges such as delays in maintenance work, miscommunication, and lack of proper updates about the status of complaints. This system provides a structured and user-friendly platform where issues can be reported with ease, tracked in real-time, and resolved more efficiently. By closing the gap between reporting and resolution, the project improves both transparency and accountability within campus management.

The project objectives have been met by creating a system that is secure, scalable, and easy to use. Users are able to register, report issues with detailed descriptions, attach relevant information, and track the status of their complaints. Administrators benefit from a centralized dashboard where issues can be viewed, categorized, prioritized, and resolved in an organized manner. Features such as authentication, role-based access, session handling, and digital records ensure that the platform remains reliable and trustworthy. The structured database also helps in maintaining a clear record of issues, making analysis and reporting much easier.

From the perspective of campus operations, this system makes a positive contribution by improving communication between students and administration. Since every issue is documented and monitored, there is less room for miscommunication, and users are assured of transparency regarding the progress of their complaints. At the same time, administrators are better equipped to allocate resources, respond to critical problems quickly, and identify recurring issues that require preventive maintenance. The system also allows institutions to make informed, data-driven decisions for future infrastructure planning.

On the technical side, the development of this project was a valuable learning experience. The use of HTML, CSS, JavaScript, and Tailwind CSS for the frontend led to the creation of a responsive and modern interface. The Flask framework with Python was used to implement backend logic such as authentication, issue management, and integration with the MySQL database. This provided practical exposure to relational databases, SQL queries, and secure data handling. Testing at various levels—including functional, performance, and security testing—helped ensure that the system worked as intended. The project also reinforced the importance of proper planning, systematic documentation, and software engineering practices.

During development, some challenges were encountered, such as designing multiple user roles,

ensuring accurate status tracking, validating inputs, and addressing security concerns. These issues were tackled through role-based access control, input validation techniques, and secure data storage methods. The modular architecture and careful database design also mean that the project can be easily extended with new features in the future.

Although the current system fulfils its main purpose, there are several opportunities for enhancement. A mobile application for Android and iOS would make the platform more accessible and convenient, with features such as offline reporting, push notifications, and camera/geolocation integration. Advanced technologies like machine learning and artificial intelligence could be used to automatically categorize issues, predict their severity, and analyse recurring trends for proactive maintenance. Integration with IoT devices could further automate the reporting process, for example, by detecting faults in real time and creating issue reports automatically. In addition, administrators could benefit from advanced analytics dashboards, multilingual support, enhanced security measures like two-factor authentication, and even gamification features to encourage user participation.

In summary, the Fix-My-Campus project demonstrates how technology can significantly improve the efficiency and responsiveness of campus management. It has shown that by combining user-friendly design with robust security and scalability, institutions can create smarter, more transparent, and more accountable systems. While the current version already delivers a reliable solution for managing campus issues, the system also lays the groundwork for future advancements such as AI, IoT, and advanced analytics. The successful completion of this project highlights the role of innovative software solutions in solving practical problems within educational institutions and underlines their potential for long-term impact.

FUTURE ENHANCEMENTS

The **Fix-My-Campus** platform has been designed to streamline reporting, tracking, and resolving campus infrastructure issues. While the current system fulfils its core objectives, there are numerous opportunities to expand its functionality, improve usability, and integrate advanced technologies. The following sections discuss possible future enhancements in detail.

1. Mobile Application Development

Currently, the platform is web-based, which may limit accessibility for students and staff who primarily use mobile devices. Developing a **mobile application** for Android and iOS can provide:

- **Push Notifications**: Instant alerts for issue updates, new announcements, or maintenance schedules.
- **Offline Reporting**: Allow users to submit issues even without internet connectivity, which are synced automatically once the connection is restored.
- **Camera Integration**: Simplify capturing and uploading images or videos of issues directly from the device.
- **Geolocation Services**: Automatically attach the location of the reported issue, helping maintenance staff identify problem areas quickly.

2. Advanced Analytics and Reporting

At present, the system only maintains a simple log of issues and their current status. In the future, this can be enhanced by adding more advanced analytics and reporting features to help administrators manage the campus more effectively.

- **Dashboards:** Interactive dashboards can display real-time information, such as the number of issues reported by category, severity, and department.
- **Trend Analysis:** By analysing past data, the system can highlight recurring problems in specific areas of the campus, allowing management to take preventive measures.
- **Predictive Maintenance:** Historical records can be used to predict potential issues and schedule maintenance work before failures occur.
- **Exportable Reports:** The system can allow administrators to generate reports formats such as PDF, Excel, which can be useful for audits and management meetings.

3. Integration with Campus Management Systems

Right now, **Fix-My-Campus** functions as an independent system. In the future, it can be integrated with other campus management platforms to create a more seamless workflow.

- **Student and Staff Databases:** Linking with existing university records can enable automatic authentication of users and proper role assignment without manual intervention.
- **Inventory Management:** By connecting with inventory systems, reported issues can be matched with available spare parts and tools, reducing delays in maintenance.
- **Scheduling Systems:** The system can also be tied to scheduling tools so that repair work is automatically assigned to staff based on availability and the urgency of the issue.

4. AI-Powered Issue Categorization

Manual categorization of issues may sometimes result in misclassification, delaying resolution. Implementing **AI and Machine Learning** can enhance efficiency:

- **Automated Categorization**: Use Natural Language Processing (NLP) to analyse issue descriptions and assign categories automatically.
- **Priority Prediction**: Machine learning models can predict the severity and urgency of issues, enabling staff to address critical problems first.
- **Image Recognition**: AI can analyse uploaded images to identify the type of issue, such as broken furniture, electrical faults, or water leakage.

5. Multi-Level User Roles and Permissions

At present, the system mainly separates users into two categories: general users and administrators. In the future, the role management feature can be extended to provide more control and flexibility.

- **Department Heads:** Can monitor and track issues that belong specifically to their departments.
- **Maintenance Staff:** Tasks can be assigned according to individual skills and workload to improve efficiency.
- **Super Admins:** Will have complete control over system settings, user management, and generation of global reports.

- **Guest Access:** Temporary users or visitors can be allowed limited access to report issues without going through the full registration process.

6. Gamification for User Engagement

Encouraging students and staff to actively report problems is crucial for the success of the system. Gamification can be used to make this process more engaging.

- Points and Badges: Users can be rewarded for reporting issues or providing detailed information.
- Leaderboards: A ranking system can highlight active users, creating healthy competition.
- Incentives: Tangible rewards such as certificates or campus-related privileges can be given to encourage consistent participation.

7. Multi-Campus and Cross-Institution Functionality

For universities or organizations that have multiple campuses, the system can be expanded to handle operations across locations.

- **Centralized Administration:** A single dashboard can be used to manage issues from multiple campuses.
- **Cross-Campus Reporting**: Helps identify recurring problems across institutions and apply standardized solutions.

8. Enhanced Security Features

- **Two-Factor Authentication (2FA):** An additional verification layer for safer login.
- **Role-Based Access Control (RBAC):** Ensures that users only access information relevant to their role.
- **Data Encryption:** Sensitive data can be encrypted both during storage and transmission.
- **Audit Logs:** Record all system activity to detect unauthorized use or suspicious behaviour.

9. Integration with IoT Devices

By using Internet of Things (IoT) devices, the system can move toward automated problem detection and reporting.

- **Smart Sensors:** Can monitor water leaks, electrical faults, or air quality in real-time.
- **Automatic Issue Creation:** Sensors can directly generate issue reports without requiring manual input.
- **Energy Management:** Track power consumption and identify unusual usage or wastage.

10. Multilingual Support

To support a wider range of users, including international students and staff, the system can include multilingual options.

- **Dynamic Language Switching:** Users can select their preferred language from within the application.
- **Localized Notifications:** Alerts and system emails will be delivered in the chosen language.
- **Accessibility:** Improves inclusivity by making the platform easier for non-native speakers.

11. Cloud Deployment and Scalability

While the current version may be hosted on local servers, cloud deployment offers a stronger and more scalable solution.

- **Cloud Hosting (AWS/Azure/GCP):** Provides high availability, reliability, and disaster recovery.
- **Auto-Scaling:** Ensures that the system can handle sudden increases in user activity during peak times.
- **Serverless Architecture:** Helps reduce operating costs while maintaining performance.

12. Feedback and Collaboration System

Adding collaboration and feedback options can make the system more interactive and improve communication between users and staff.

- **Commenting on Issues:** Users and staff can post updates, solutions, or discussions on reported issues.
- **Feedback Mechanism:** After an issue is resolved, users can rate the quality and timeliness of the resolution.
- **Knowledge Base:** A repository of common issues and their solutions can be maintained to help in resolving problems faster.

BIBLIOGRAPHY

[1] I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill Education, 2020.

[3] L. Welling and L. Thomson, PHP and MySQL Web Development, 5th ed. Addison-Wesley, 2017.

[4] D. Flanagan, JavaScript: The Definitive Guide, 7th ed. O'Reilly Media, 2020.

[5] Flask Documentation, 2025. [Online]. Available: https://flask.palletsprojects.com

[6] MySQL Documentation, 2025. [Online]. Available: https://dev.mysql.com/doc/

[7] Mozilla Developer Network (MDN), 2025. HTML, CSS, JavaScript Guides. [Online]. Available: https://developer.mozilla.org

[8] Tailwind CSS Documentation, 2025. [Online]. Available: https://tailwindcss.com/docs

[9] M. Jaiswal and A. Singh, "Design and Implementation of Campus Issue Tracking System," International Journal of Computer Applications, vol. 178, no. 12, pp. 1–7, 2019.

[10] P. Chandrasekaran and S. Kumar, "Web-Based Maintenance Management Systems in Educational Institutions," International Journal of Advanced Research in Computer Science, vol. 9, no. 5, pp. 22–29, 2018.

[11] A. Grover and R. Sharma, "Role of Database Management Systems in Modern Web Applications," Journal of Software Engineering and Applications, vol. 13, no. 3, pp. 2020.

[12] J. Nielsen, Usability Engineering, Morgan Kaufmann, 2022.

[13] R. Patel and K. Mehta, "IoT-Enabled Campus Management Solutions: Opportunities and Challenges," International Journal of Advanced Research in Engineering and Technology, vol. 12, no. 4, pp. 105–115, 2021.

[14] W3Schools, 2025. HTML, CSS, JavaScript, SQL Tutorials. [Online]. Available: https://www.w3schools.com

[15] Stack Overflow, 2025. Community Solutions and Discussions for Web Development Issues. [Online]. Available: https://stackoverflow.com

[16] IEEE Standards Association, "IEEE Standard for Software Testing and Quality Assurance," IEEE Xplore Digital Library, 2019.

[17] S. Gupta and N. Verma, "Security and Authentication in Web Applications," International Journal of Computer Science Trends and Technology, vol. 8, no. 1, pp. 2020.

SIMILARITY REPORT

DrillBit

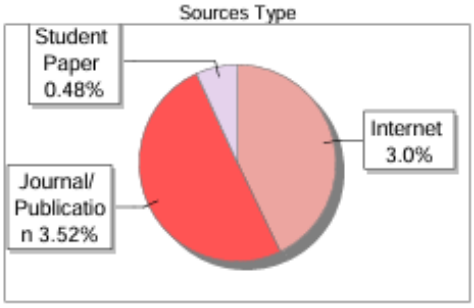
The Report is Generated by DrillBit Plagiarism Detection Software

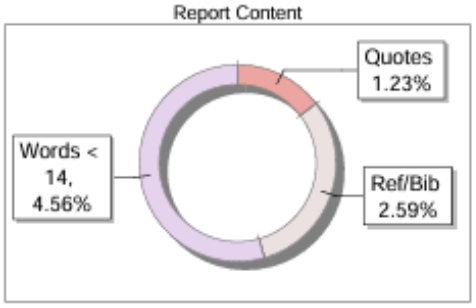
Submission Information

Author Name	prasanna
Title	fix my campus
Paper/Submission ID	4425689
Submitted by	chaithrars@sjbit.edu.in
Submission Date	2025-09-27 10:16:37
Total Pages, Total Words	79, 10968
Document type	Project Work

Result Information

Similarity **7 %**





Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File